RandomDataStreams.jl

October 18, 2026

Contents

Contents ii

I Home 1

1 RandomDataStreams.jl Documentation 2

1.1 Contents . . . 2

1.2 Quick example . . . 3

II Getting Started 4

2 Getting Started 5

2.1 Installation . . . 5

2.2 Choosing a generator . . . 5

2.3 Seeding: one rule for every generator . . . 5

2.4 First numbers . . . 6

2.5 Supported output types . . . 7

2.6 Reproducibility . . . 7

2.7 Full Random API . . . 7

2.8 Next steps . . . 8

III Streams & Substreams 9

3 Streams & Substreams 10

3.1 Why streams? The case for Common Random Numbers (CRN) . . . 10

3.2 The two object families . . . 12

3.3 Producing independent streams . . . 12

3.4 Threads . . . 12

3.5 Distributed . . . 13

3.6 Navigating substreams . . . 14

3.7 Saving, restoring, jumping . . . 14

3.8 Normal variates and variance reduction . . . 15

3.9 Scope: host-side streams, device-side draws . . . 16

3.10 One interface, every generator . . . 17

3.11 Guarantees . . . 18

IV API Reference 19

4 API Reference 20

4.1 Types . . . 20

4.2 The common interface . . . 20

4.3 MRG32k3a . . . 22

4.4 MRG63k3a . . . 23

4.5 Xoshiro256+ . . . 24

4.6 Xoshiro / xoroshiro families . . . 25

4.7 Where the families actually differ . . . 25

4.8 PCG . . . 25
4.9 Counter-based generators . . . 27

V Docstrings **30**
5 Docstrings **31**
5.1 Abstract types . . . 31
5.2 MRG32k3a . . . 32
5.3 MRG63k3a . . . 33
5.4 Xoshiro / xoroshiro families . . . 33
5.5 Counter-based generators . . . 37
5.6 Normal and exponential variates . . . 38

VI Implementation Notes **49**
6 Implementation Notes **50**
6.1 MRG32k3a . . . 50
6.2 MRG63k3a . . . 51
6.3 Xoshiro256+ . . . 53
6.4 PCG . . . 54
6.5 Counter-based generators . . . 55
6.6 Integer outputs of MRG32k3a . . . 55
6.7 Floating-point outputs . . . 56
6.8 Normal and exponential variates . . . 56
6.9 Testing strategy . . . 57
6.10 Performance snapshot . . . 57

VII Validation **61**
7 Statistical Validation (TestU01) **62**
7.1 In the test suite: SmallCrush . . . 62
7.2 Dependence between streams . . . 62
7.3 The integer paths, at the bit level . . . 63
7.4 Running Crush and BigCrush . . . 64
7.5 Reading a summary report . . . 65
7.6 Long campaigns: running for days without a session . . . 69
7.7 How the batteries are driven . . . 73

VIII Generator Comparison **75**
8 Choosing a generator **76**
8.1 Recurrence-based generators . . . 76
8.2 Counter-based generators . . . 76
8.3 When to choose MRG32k3a . . . 76
8.4 When to choose MRG63k3a . . . 77
8.5 When to choose a xoshiro/xoroshiro variant . . . 78
8.6 When to choose PCG64 . . . 78
8.7 When to choose Philox or Threefry . . . 79
8.8 Scrambler choice within a xoshiro family . . . 79
8.9 Related work in the Julia ecosystem . . . 79
8.10 Practical notes . . . 80

IX FAQ **81**
9 FAQ **82**
9.1 Why is an all-zero state forbidden? . . . 82
9.2 Are MRG integer outputs uniform over their full width? . . . 82

9.3 MRG32k3a or MRG63k3a? . . . 82
9.4 Why doesn't `sample(v, k)` work with RandomDataStreams generators? . . . 82
9.5 Where did `srand`, `next_stream`, `short_jump`, `long_jump` go? . . . 83
9.6 What does an integer seed mean? . . . 83
9.7 Can `rand` return 0 or 1? . . . 83
9.8 How do I run truly parallel simulations? . . . 83
9.9 Which generator should I pick? . . . 83
9.10 What does "counter-based" buy me? . . . 83
9.11 Philox or Threefry? . . . 84
9.12 Is it safe to use 1, 2, 3, ... as counter-based stream keys? . . . 84
9.13 Can I reproduce cuRAND's Philox and MRG32k3a streams? . . . 84
9.14 Can I reproduce a NumPy pipeline? . . . 84
9.15 Why can't I choose PCG's increment to get more streams? . . . 85
9.16 Does this package run on the GPU? . . . 85
9.17 Why does `randn` use inversion rather than the Ziggurat? . . . 85

Part I

Home

Chapter 1

RandomDataStreams.jl Documentation

Streamable pseudo-random number generators for Julia, with non-overlapping streams and substreams:

- **MRG32k3a** (L'Ecuyer et al. 2002) — the reference combined MRG of the simulation literature;
- **MRG63k3a** (L'Ecuyer 1999) — the same construction in 64-bit arithmetic: period ≈ 2^377, and a step that yields 63 random bits instead of 32;
- the **xoshiro / xoroshiro** families (Blackman & Vigna) — 128, 256 and 512 bits of state, three scramblers each;
- **PCG64** and **PCG64DXSM** (O'Neill) — the default bit generator of NumPy, matched bit for bit, with streams from the closed-form LCG jump;
- the **counter-based** generators **Philox** and **Threefry** (Salmon et al.

  1. — streams are keys, and any draw can be recomputed from its index.

All of them share one interface: the same stream and substream navigation, the same state contract, and the full `Random` API.

1.1 Contents

1. Getting Started
2. Streams & Substreams
3. API Reference
4. Docstrings
5. Implementation Notes
6. Validation
7. Generator Comparison
8. FAQ

1.2 Quick example

```julia
using RandomDataStreams

gen  = MRG32k3aGen()
rngA = next_stream!(gen)      # independent stream A
rngB = next_stream!(gen)      # independent stream B

rand(rngA)                   # Float64 in (0, 1)
next_substream!(rngA)        # move to the next substream of A
reset_stream!(rngA)          # rewind stream A to its beginning
```

The same code runs on any generator the package ships — only the first line changes:

```julia
gen = Philox4x64Gen()        # or Xoshiro256plusGen(), Threefry4x64Gen(), ...
```

Part II

Getting Started

Chapter 2

Getting Started

RandomDataStreams.jl provides pseudo-random number generators with support for **non-overlapping streams and substreams**, following the stream/substream model of L'Ecuyer et al. (2002).

2.1 Installation

```
using Pkg
Pkg.add("RandomDataStreams")
```

or, from a local clone:

```
using Pkg
Pkg.develop(path = "path/to/RandomDataStreams.jl")
```

Requirements: Julia ≥ 1.9. The only dependency is the standard library `Random`; CUDA.jl, if loaded, enables GPU fills for the counter-based generators (see Streams & Substreams).

2.2 Choosing a generator

RandomDataStreams.jl implements the full xoshiro/xoroshiro family of Blackman & Vigna, L'Ecuyer's MRG32k3a and MRG63k3a, O'Neill's PCG (the default bit generator of NumPy), and the Philox and Threefry CBRNGs. All variants of a xoshiro family share the same linear transition and jump constants; only the output scrambler differs.

See Generator Comparison for guidance and benchmarks.

2.3 Seeding: one rule for every generator

Whatever generator you pick, there are three ways to seed it, and they mean the same thing in every family:

```
MRG32k3aGen()             # the package default seed, 12345
MRG32k3aGen(20260830)     # an integer seed
MRG32k3aGen([1,2,3,4,5,6])# the family's own seed vector

Xoshiro256ppGen(20260830) # exactly the same three forms
PCG64Gen(20260830)
PhiloxGen(20260830)
```

Type	Family	State	Period	Scrambler
`PhiloxRNG`	Philox4x32-10	128-bit counter	2^130	Feistel-like network
`Philox4x64RNG`	Philox4x64-10	128-bit counter	2^130	Feistel-like network
`Threefry4x64RNG`	Threefry4x64-20	128-bit counter	2^130	Threefish rounds (add/rot/xor)
`Threefry4x32RNG`	Threefry4x32-20	128-bit counter	2^130	Threefish rounds (add/rot/xor)
`Xoroshiro128p`	xoroshiro128	128 bits	2^128 − 1	`s0 + s1`
`Xoroshiro128ss`	xoroshiro128	128 bits	2^128 − 1	high bits, **
`Xoroshiro128pp`	xoroshiro128	128 bits	2^128 − 1	high bits, ++
`Xoshiro256p`	xoshiro256	256 bits	2^256 − 1	`s0 + s3`
`Xoshiro256ss`	xoshiro256	256 bits	2^256 − 1	high bits, **
`Xoshiro256pp`	xoshiro256	256 bits	2^256 − 1	high bits, ++
`Xoshiro512p`	xoshiro512	512 bits	2^512 − 1	`s0 + s2`
`Xoshiro512ss`	xoshiro512	512 bits	2^512 − 1	high bits, **
`Xoshiro512pp`	xoshiro512	512 bits	2^512 − 1	high bits, ++
`MRG32k3a`	combined MRG (L'Ecuyer)	192 bits	≈ 2^191	native `Float64`, 32-bit step
`MRG63k3a`	combined MRG (L'Ecuyer)	384 bits	≈ 2^377	native `Float64`, 63-bit step
`PCG64`	PCG (O'Neill)	128 bits	2^128	XSL-RR permutation
`PCG64DXSM`	PCG (O'Neill)	128 bits	2^128	DXSM permutation

An **integer** is a *seed*: it is expanded through splitmix64 and folded into whatever the family accepts, so `T(20260830)` is equivalent to `Random.seed!(T(), 20260830)` everywhere. A value in the family's **own representation** — its seed vector, or a `UInt128` for PCG — is taken as the state or key itself, not hashed.

The same three forms work on the stream types directly (`MRG32k3a(20260830)`, `Xoshiro256pp(20260830)`, …). See Streams & Substreams for the full interface table.

2.4 First numbers

```julia
using RandomDataStreams

rng = MRG32k3a()      # default seed [12345, ..., 12345]
rand(rng)             # 0.12701112204657714
rand(rng)             # 0.3185275653967945
```

```julia
x = Xoshiro256p(UInt64[1, 2, 3, 4])
rand(x)               # Float64 in (0, 1)
```

2.5 Supported output types

MRG32k3a

- `Float64` (native), plus `Float32`, `Float16` via conversion
- `UInt8`, `UInt16`, `UInt32`, `UInt64`, `UInt128`
- `Int8`, `Int16`, `Int32`, `Int64`, `Int128` (same bit patterns)
- `Bool`
- Ranges: `rand(rng, 1:10)` works through the standard `Random` machinery

Note: integer outputs of MRG32k3a are built from its native ~32-bit resolution, and MRG63k3a's from its ~63-bit one; they are not uniform over their full width. Use them for indices, choices and flags rather than cryptography.

Xoshiro256p

- `Float64` (native path), `Float32`, `Float16`, all in the open interval `(0, 1)` — see Implementation Notes
- `UInt64`
- Ranges: `rand(rng, 1:10)` works through the standard `Random` machinery

2.6 Reproducibility

Every generator accepts an explicit seed:

```
rng = MRG32k3a([42, 1, 2, 3, 4, 5])
x   = Xoshiro256p(UInt64[0xdead, 0xbeef, 0xcafe, 0xbabe])
```

Seeds are validated by checkseed, or checkseed63 for MRG63k3a: a valid seed of either has 6 non-negative values; the first three must be < m1 and not all zero, the last three < m2 and not all zero.

The standard Julia interface also works with every generator:

```
Random.seed!(rng, 42)          # integer seed (splitmix64 expansion)
Random.seed!(rng, [7,7,7,8,8,8])   # explicit MRG32k3a seed (validated)
```

2.7 Full Random API

RandomDataStreams generators are drop-in substitutes for Julia's built-in RNGs: anywhere a function accepts an `AbstractRNG`, you can pass an RandomDataStreams generator and keep your stream/substream control.

```
using RandomDataStreams, Random

rng = next_stream!(MRG32k3aGen())     # any RandomDataStreams generator works here

rand(rng, 5)                          # Vector{Float64}, 5 draws
A = rand(rng, Float64, 2, 3)          # 2x3 matrix
z = randn(rng)                        # standard normal, by inversion
e = randexp(rng)                      # exponential, by inversion
```

```
v = shuffle(rng, collect(1:8))        # shuffled copy
p = randperm(rng, 6)                  # random permutation
buf = Vector{Float64}(undef, 3)
rand!(rng, buf)                       # fill in place
Random.randstring(rng, 10)            # random string
```

Seeding through the standard interface makes results reproducible:

```
Random.seed!(rng, 42)
u1 = [rand(rng) for _ in 1:3]
Random.seed!(rng, 42)
u2 = [rand(rng) for _ in 1:3]
u1 == u2                              # true
```

Only `sample` requires StatsBase.jl and is out of scope.

2.8 Next steps

- Streams & substreams
- API reference
- Implementation notes

Part III

Streams & Substreams

Chapter 3

Streams & Substreams

The central abstraction of RandomDataStreams.jl is the *stream*: a long, non-overlapping subsequence of a generator's period. Streams can themselves be split into *substreams*. This is the architecture recommended by L'Ecuyer et al. (2002) for parallel and replicated stochastic simulation:

- **Streams** are handed to independent replications or parallel workers; their outputs never overlap, so replications are statistically independent.
- **Substreams** delimit scenarios *within* one replication; rewinding a substream while keeping the scenario structure enables techniques such as common random numbers.

3.1 Why streams? The case for Common Random Numbers (CRN)

The whole point of the stream/substream architecture is **variance reduction when comparing systems** — arguably the most important technique in stochastic simulation design (L'Ecuyer et al. 2002; Law, *Simulation Modeling and Analysis*).

Suppose you compare K configurations of a stochastic system (two inventory levels, two queue disciplines...). The naive approach runs each configuration with *independent* random numbers. The variance of the estimated difference is then

```
Var[D̂] = Var[Ȳ₁]/n + Var[Ȳ₂]/n
```

With **common random numbers**, every configuration of a given replication consumes the *same* underlying uniforms: the per-replication differences are strongly positively correlated and

```
Var[D̂_CRN] = Var[Ȳ₁]/n + Var[Ȳ₂]/n − 2·Cov[Ȳ₁, Ȳ₂]/n
```

— often orders of magnitude smaller. But CRN only works if the random number usage can be *synchronised exactly* across configurations, replication after replication. That is precisely what RandomDataStreams's machinery guarantees:

Measurable example: comparing two inventory levels

Daily cost of an inventory system over 30 days (demands N(100, 20) truncated at 0), comparing stock level s = 50 vs s = 60:

Need	RandomDataStreams tool
Replications must be independent	a fresh stream per replication (`next_stream!`)
Configurations must see identical randomness	rewind each configuration to the same substream start (`reset_stream!` / `reset_substream!`)
No accidental overlap between replications	streams are provably disjoint

```
using RandomDataStreams, Statistics

function cost(s, rng)
    total, stock = 0.0, Float64(s)
    for _ in 1:30
        stock = min(stock + 40, 200)
        d = max(0.0, 100 + 20randn(rng))
        sold = min(stock, d)
        total += 5(d - sold) + 0.1max(0, stock - sold - 10)
        stock -= sold
    end
    total
end

function replications(crn::Bool, n::Int)
    gen = MRG32k3aGen()
    diffs = Vector{Float64}(undef, n)
    for i in 1:n
        r50 = next_stream!(gen)
        l50 = cost(50, r50)
        r60 = crn ? reset_stream!(copy(r50)) : next_stream!(gen)
        l60 = cost(60, r60)
        diffs[i] = l60 - l50
    end
    return diffs
end

for crn in (true, false)
    ds = replications(crn, 4_000)
    println("CRN = \$crn: mean diff ≈ \$(round(mean(ds), digits=1)), ",
            "var(diff) ≈ \$(round(var(ds), digits=1))")
end
```

Typical output:

```
CRN = true:  mean diff ≈ -49.9, var(diff) ≈ 0.1
CRN = false: mean diff ≈ -39.2, var(diff) ≈ 540775.9
```

The variance of the comparison drops by **six orders of magnitude**: with CRN, both policies face identical demand sequences, so the observed difference measures the effect of the policy change rather than the noise of unrelated demand scenarios. Without CRN you would need ~10^6 more replications for the same precision.

This example uses `copy(r50)` + `reset_stream!`: both policies replay the very same stream from its beginning. `reset_substream!` generalises the pattern when each replication itself contains several scenarios.

3.2 The two object families

Every generator in the package comes as a pair, and the distinction is the one the whole interface rests on.

1. **Generator objects** (`<: AbstractRNGStream`) are factories. They hold the seed of the *next* stream and hand out streams with `next_stream!`. They produce no random numbers themselves.
2. **Streams** (`<: AbstractStreamableRNG`) produce the numbers, and move along the stream and substream hierarchy. They are `AbstractRNG`s, so they work anywhere Julia's own generators do.

Family	Generator object	Stream
MRG32k3a	`MRG32k3aGen`	`MRG32k3a`
MRG63k3a	`MRG63k3aGen`	`MRG63k3a`
xoroshiro128	`Xoroshiro128pGen`, `Xoroshiro128ssGen`, `Xoroshiro128ppGen`	`Xoroshiro128p`, `Xoroshiro128ss`, `Xoroshiro128pp`
xoshiro256	`Xoshiro256pGen`, `Xoshiro256ssGen`, `Xoshiro256ppGen`	`Xoshiro256p`, `Xoshiro256ss`, `Xoshiro256pp`
xoshiro512	`Xoshiro512pGen`, `Xoshiro512ssGen`, `Xoshiro512ppGen`	`Xoshiro512p`, `Xoshiro512ss`, `Xoshiro512pp`
PCG	`PCG64Gen`, `PCG64DXSMGen`	`PCG64`, `PCG64DXSM`
Philox	`PhiloxGen` (4x32-10), `Philox4x64Gen`	`PhiloxRNG`, `Philox4x64RNG`
Threefry	`Threefry4x32Gen`, `Threefry4x64Gen`	`Threefry4x32RNG`, `Threefry4x64RNG`

`Xoshiro256plusGen` is an alias of `Xoshiro256pGen`, kept from earlier versions.

The interface is the same across the table: whatever the family, a generator object answers to `next_stream!`, `srand!`, `get_state` and `set_state!`, and a stream answers to `next_substream!`, `reset_substream!`, `reset_stream!`, `advance_state!`, `get_state`, `set_state!` and `srand!`. Code written against the two abstract types never names a family — see the API reference.

3.3 Producing independent streams

```julia
using RandomDataStreams

gen = MRG32k3aGen()             # or Xoshiro256plusGen(UInt64[...])

worker1 = next_stream!(gen)       # stream 1
worker2 = next_stream!(gen)       # stream 2 — guaranteed disjoint from stream 1
worker3 = next_stream!(gen)       # stream 3 — ...
```

Each call to `next_stream!` advances the generator's internal seed by a huge leap (2^127 values for MRG32k3a, 2^250 for MRG63k3a, a full `long_jump!` for Xoshiro256+), which is what guarantees non-overlap.

3.4 Threads

Streams share no state, so drawing from different streams on different threads needs no synchronisation. Take one stream per replication first, then parallelise over the replications:

```julia
using RandomDataStreams, Base.Threads
```

```
nrep = 1_000
gen  = MRG32k3aGen()                    # any generator object
rngs = next_stream!(gen, nrep)          # one stream per replication, serially

results = Vector{Float64}(undef, nrep)
@threads for r in 1:nrep
    rng = rngs[r]                       # replication r owns stream r
    results[r] = sum(rand(rng) for _ in 1:10^5)
end
```

The results are those of the same streams drawn one after another, which the test suite checks for every family.

Index the streams by the unit of work, not by the thread. Replication `r` then draws from stream `r` whatever the number of threads and however the scheduler hands out iterations, so `results` is identical with `julia -t 1` and `julia -t 64` — the requirement Passerat-Palmbach, Mazel & Hill (2012) state for GPU threads, and the one that makes a run reproducible on another machine. Taking `nthreads()` streams and splitting the work between them would not have this property: a replication's draws would depend on how many threads shared the work. Nor should a stream be chosen by `threadid()`, which is not stable while a task runs. When replications number in the millions, give each stream a fixed-size batch of them: what matters is that the batch, not the thread, determines the stream.

Do not call ‘next_stream¡ inside the parallel loop

A generator object holds the seed of the next stream and rewrites it on every call. Concurrent calls read the same seed, so the streams handed out overlap — and nothing reports it. In a measurement here, 64 threads calling `next_stream!` on one shared generator received between 57 and 63 distinct streams instead of 64, on three consecutive runs. The failure is silent and destroys precisely the guarantee this package exists to provide.

`next_stream!(gen, n)` exists so that the correct pattern is also the shorter one.

3.5 Distributed

Separate processes cannot share stream objects, so the seeds travel instead. Take them from the generator before each `next_stream!`, and rebuild the stream on the worker with `srand!`:

```
using Distributed
@everywhere using RandomDataStreams

gen   = MRG32k3aGen()
seeds = Vector{Any}(undef, nworkers())
for i in 1:nworkers()
    seeds[i] = get_state(gen)     # seed of the stream about to be produced
    next_stream!(gen)             # advance past it
end

@sync for (i, w) in enumerate(workers())
    seed = seeds[i]
    @spawnat w begin
        rng = MRG32k3a(1)         # any valid seed; replaced on the next line
        srand!(rng, seed)         # this worker's stream, from its beginning
        # ... draw from rng ...
    end
end
```

Use `srand!` here, not `set_state!`. They are not interchangeable: `set_state!` moves the current position only, leaving the stream and substream boundaries where the constructor put them, so a later `reset_stream!` on the worker would rewind to the wrong place. `srand!` sets all three checkpoints, which is what makes the worker's stream a stream rather than a position in someone else's.

Both are portable across families, as is `get_state` on a generator object; the representation that travels is opaque, so hand it back unchanged rather than interpreting it.

3.6 Navigating substreams

Within a single RNG, three functions move along the stream/substream hierarchy (all return the modified RNG):

Function	Effect
`reset_stream!(rng)`	jump to the very beginning of the current stream
`reset_substream!(rng)`	jump to the beginning of the current substream
`next_substream!(rng)`	advance to the next substream

Example — common random numbers across two scenarios:

```
rng = next_stream!(MRG32k3aGen())

rand(rng)                # consume some variates of substream 1...
rand(rng)

next_substream!(rng)     # switch to substream 2 for scenario B
uB = rand(rng)

reset_stream!(rng)       # replay substream 1 from scratch for scenario A
uA = rand(rng)           # same underlying uniforms as the first draw
```

For Xoshiro256+, `next_substream!` is implemented as a `short_jump!` (2^96-value leap) and `reset_stream!`/`next_substream!` manipulate the saved `Bg`/`Ig` checkpoints.

3.7 Saving, restoring, jumping

```
state = get_state(rng)     # a copy; safe to store
# ... consume numbers ...
```

To restore an MRG32k3a or MRG63k3a state, rebuild the generator with the triple constructor (`Cg`, `Bg`, `Ig` all set to the snapshot):

```
snapshot = get_state(rng)
xs = [rand(rng) for _ in 1:5]
clone = MRG32k3a(snapshot, snapshot, snapshot)
rand(clone) == xs[1]       # true — resumes exactly after the snapshot
```

Both MRG generators additionally support arbitrary jumps inside a stream:

```
advance_state!(rng, e, c)
```

moves the state forward by n steps, where n = 2^e + c (e may be negative, and negative c moves backwards). This costs O(log n) matrix operations, independent of the distance jumped.

3.8 Normal variates and variance reduction

Every generator draws its normals by inversion: `randn(rng)` is `norminv(rand(rng, Float64))`, one uniform per normal, through the standard normal quantile function Φ^{-1}. Exponential variates follow the same rule, `randexp(rng)` being `-log(1 - rand(rng, Float64))`. The standard library uses the Ziggurat for both instead, which is faster, but which breaks what common random numbers, antithetic variates and randomized quasi-Monte Carlo need from the map between uniforms and normals:

	Ziggurat (the standard library's `randn`)	Box-Muller (`randn_boxmuller!`, GPU)	Inversion (`randn`, `randn!`)
Uniforms per normal	variable: a rejection now and then	2 → 2, coupled	1 → 1
Antithetic variates (u → 1 − u)	no −z: the sign, the layer and the abscissa come from different bits of one word	no −z: the radius changes, and cos 2π(1 − u) = cos 2πu	z → −z, exactly on the word generators' grid
Common random numbers	hold only while both systems make the same calls in the same order: how many words a normal takes depends on their values	hold, in pairs: a normal's partner is drawn with it	natural: the k-th normal depends on the k-th uniform alone
Randomized quasi-Monte Carlo	incompatible: rejection makes the dimension random	valid, but each normal depends on two coordinates, which weakens the reduction	the reference method: one coordinate per normal
k-th normal computable directly	no	in pairs	yes — which is what makes the GPU fill the CPU's

The antithetic identity is exact, not approximate. The word generators' uniforms are odd multiples of 2^{-53}, a grid symmetric about $1/2$ on which `1 - u` and `u - 1/2` are both exact, and `norminv` works from $|u-1/2|$ and its sign, so `norminv(1 - u) == -norminv(u)` for every uniform they return, the extremes included. An antithetic stream of uniforms therefore gives exactly the negated normals. `test/test_randn_variance_reduction.jl` checks each cell of the antithetic row, for the three methods.

How far the tails reach is set by the resolution of the uniform, since the extreme normal is Φ^{-1} of the extreme uniform. On the 53-bit grid of the word generators that is ±8.21. MRG32k3a's uniforms are multiples of $1/(m_1+1)$, about 2.3×10^{-10}, so its normals stay within ±6.23; MRG63k3a reaches −9.00 below and 8.21 above, where `Float64` runs out of resolution near 1. A `Float32` or `Float16` normal from `randn` is the `Float64` one rounded, as in the standard library, so it keeps these tails. `randn32` is the other trade: the single-precision Φ^{-1} of a `Float32` uniform, in `Float32` arithmetic throughout, whose tails stop at ±5.29 — the quantile of the `Float32` grid's extreme 2^{-24}, with the probability beyond it, about 10^{-7}, landing on that last point. It keeps every property in the table, the exact antithetic identity included, on the `Float32` grid.

What it costs is speed: Φ^{-1} is a rational function of degree 7, and a logarithm in the tails, where the Ziggurat is mostly a table lookup and a multiplication. On one core of an Intel Core Ultra 9 285, measured while the other cores were busy, a normal from Xoshiro256++ took about 4.5 ns more than the uniform under it, against 1.2 ns for the Ziggurat; for the counter-based generators and the MRGs, whose uniforms cost more, the two came out within 15% of each other. On a GPU the map runs in `Float64`, which a consumer card computes at a

small fraction of its Float32 rate. An RTX A1000 fills about 0.6 billion normals a second by inversion, Float64 or Float32, against about 1.4 billion Float64 and 25 to 30 billion Float32 normals for randn_boxmuller! — and 1.6 billion Float64 for cuRAND's own Box-Muller, which meets the same Float64 limit. randn32! brings inversion to about 11 billion Float32 normals a second there (Philox4x32), with the CPU's values on the device as for randn!. On that card, power-limited at 50 W, these rates vary by 10 to 20% from one run to the next. For uses that need none of the properties above, Julia's own Xoshiro keeps the Ziggurat, and randn_boxmuller! stays on the GPU.

To map uniforms of your own the same way — the points of a randomized quasi-Monte Carlo rule, for instance — call norminv on them. It is the same function, and gives the same bits on a GPU; the Implementation Notes explain how.

3.9 Scope: host-side streams, device-side draws

The stream object of L'Ecuyer et al. (2002) is stateful by construction — a current position, a substream anchor, a stream anchor, mutated in place. That makes it a **host-side** abstraction. A GPU kernel wants the opposite: no state at all, one value computed from the thread index. The counter-based generators reconcile the two, and they are the only ones the package runs on a device.

Counter-based generators: the GPU fill is the CPU fill. With CUDA.jl loaded, rand!(rng, A) on a CuArray{Float64}, CuArray{Float32} or CuArray{Float16} fills A on the device for every counter-based generator — Philox4x32, Philox4x64, Threefry4x32, Threefry4x64 — and gives bit for bit what the same call gives on the CPU, from any position, mid-block included. The stream object stays on the host and is left exactly where the CPU fill would leave it, so the next draw, on either device, continues the same sequence:

```
using RandomDataStreams, CUDA

rng = next_stream!(Philox4x64Gen(2026))
A = CUDA.zeros(Float32, 10^8)
rand!(rng, A)          # on the device: the values rand! gives on the CPU
rand(rng)              # on the host: the draw that follows them
```

Element k is a pure function of the key, the counter and k — the same words, in the same order, through the same open01 conversion as the stream object — so the result does not depend on the number of threads or on the launch configuration. Each thread computes one block of the generator, once, and writes every draw whose words lie in it. The conversion's one floating-point operation is a subtraction whose exact result is representable, which IEEE 754 fixes to the last bit on any device. The test suite checks the position-addressed draws against the stream object on the CPU, and the device fill against the CPU fill whenever a CUDA device is present.

Normal and exponential variates follow. randn! and randexp! on a CuArray fill it with the CPU's values, bit for bit, for the same four generators and from any position: element k is norminv (or -log(1 - u)) of the k-th Float64 uniform, and the package computes both maps with correctly rounded operations only, never CUDA's libdevice — see Normal variates and variance reduction. randn_boxmuller! (Philox4x32 only) is the faster alternative; it continues the same stream, but its values are not the CPU's.

Writing your own kernel: the addressing scheme is the work assignment. The key names the stream, the high half of the counter the substream, the low half the position. Any draw can therefore be recomputed from (key, substream, index) alone, with no object, which is exactly what a kernel needs:

```
using RandomDataStreams
const RDS = RandomDataStreams
```

```
function direct_word(key::NTuple{2,UInt32}, substream::Integer, i::Integer)
    block, word = divrem(i, 4)                    # four 32-bit words per block
    ctr = (UInt128(substream) << 64) | UInt128(block)
    c = (ctr % UInt32, (ctr >> 32) % UInt32, (ctr >> 64) % UInt32, (ctr >> 96) % UInt32)
    return RDS.philox(c, key)[word + 1]
end
```

This agrees with the stream object draw for draw; the test suite checks it, so host and device address the same sequence. philox4x32_10, philox4x32_counter and open01 are exported for this: pure, allocation-free functions that run inside a kernel, where the test suite also runs them.

Recurrence-based generators: the host computes the starting points. There is no stateless form here, and the standard pattern runs the other way: use next_stream! on the host to produce as many non-overlapping starting states as there are tasks, ship one to each, and let each iterate its own recurrence.

```
gen = Xoshiro256plusGen(UInt64[1, 2, 3, 4])
seeds = [get_state(next_stream!(gen)) for _ in 1:nworkers]    # non-overlapping
```

The jump machinery is what makes this cheap — matrix jumps for MRG32k3a, GF(2) polynomial jumps for the xoshiro families — and it is the construction L'Ecuyer et al. (2021, Sec. 2) describe for parallel environments. The package provides no device-side draws for these families: a GPU fill of one stream would need every thread to jump to its own position first.

3.10 One interface, every generator

The whole point of the two object families is that code written against them does not name a generator. Every family in the package answers to the same calls, with the same meanings — the table below is asserted for all seventeen generators in the test suite, not just documented.

On the generator object:

call	meaning
`Gen()`	seeded with the package default, 12345
`Gen(12345)`	seeded with an integer
`Gen(v)`	seeded with the family's own seed vector
`next_stream!(gen)`	the next non-overlapping stream
`srand!(gen, seed)`	reset the seed the next `next_stream!` will use
`get_state(gen)`, `set_state!(gen, s)`	save and restore it

On the stream:

call	meaning
`T()`, `T(12345)`, `T(v)`	the same three seeding forms
`next_substream!`, `reset_substream!`, `reset_stream!`	navigation; each returns the generator
`advance_state!(rng, e, c)`	move by `2^e + c` draws, negative allowed
`get_state`, `set_state!`	save and restore the position only
`srand!`, `Random.seed!`	reseed, resetting both boundaries
`copy`, `show`, the full `Random` API	as for any `AbstractRNG`

The seeding rule. A value in the family's own representation — its seed vector, or a `UInt128` for PCG — *is* the state or key. Any other integer is a *seed*, expanded through splitmix64 and folded into whatever the family

accepts. So `MRG32k3aGen(12345)`, `Xoshiro256ppGen(12345)`, `PhiloxGen(12345)` and `PCG64Gen(12345)` all mean the same kind of thing, and `T(12345)` is equivalent to `Random.seed!(T(), 12345)` everywhere.

What is not portable. `short_jump!` and `long_jump!` are the xoshiro and PCG spelling of a jump by exactly the substream and stream distance. Use `next_substream!` in code meant to work with any generator. `long_jump!` has no meaning at all for a counter-based generator, where moving to another stream means taking another key — an operation that belongs to the generator object.

3.11 Guarantees

- Streams produced by successive calls to `next_stream!` on the same generator object are **provably non-overlapping**.
- Substreams likewise partition a stream into disjoint blocks (length ≈ 2^76 steps for MRG32k3a, 2^150 for MRG63k3a).

Part IV

API Reference

Chapter 4

API Reference

All names below are exported by the `RandomDataStreams` module unless marked *(internal)*.

4.1 Types

`AbstractRNGStream` — supertype of stream *generators*: objects that mint new independent streams. `MRG32k3aGen`, `MRG63k3aGen`, the nine `Xoshiro*Gen`/`Xoroshiro*Gen`, `PCG64Gen`, `PCG64DXSMGen`, `PhiloxGen`, `Philox4x64Gen`, `Threefry4x32Gen`, `Threefry4x64Gen`.

`AbstractStreamableRNG <: Random.AbstractRNG` — supertype of usable RNGs that navigate streams and substreams. `MRG32k3a`, `MRG63k3a`, the nine xoshiro/xoroshiro variants, `PCG64`, `PCG64DXSM`, `PhiloxRNG`, `Philox4x64RNG`, `Threefry4x32RNG`, `Threefry4x64RNG`.

4.2 The common interface

Everything in this section works on **every** generator the package ships, with the same meaning. It is asserted for all seventeen in the test suite, so code written against `AbstractStreamableRNG` never has to name a family. The per-family sections below add only what is specific.

Seeding

Form	Meaning
`T()`, `Gen()`	the package default seed, 12345
`T(12345)`, `Gen(12345)`	an **integer seed**, expanded through splitmix64
`T(v)`, `Gen(v)`	the family's **own representation** — its seed vector, or a `UInt128` for PCG — taken as the state or key itself
`Random.seed!(rng, 12345)`	same as `T(12345)`, in place
`srand!(rng, seed)`, `srand!(gen, seed)`	reseed, resetting both boundaries

The distinction is the whole rule: an integer is a *seed* and gets hashed; a value shaped like the state *is* the state.

On the stream

`get_state` returns a family-specific representation. Treat it as opaque and only ever hand it back to `set_state!`.

Call	Meaning
`next_substream!(rng) -> rng`	move to the start of the next substream
`reset_substream!(rng) -> rng`	rewind to the start of the current substream
`reset_stream!(rng) -> rng`	rewind to the start of the current stream
`advance_state!(rng, e, c) -> rng`	move by `2^e + c` draws; negative moves backwards
`get_state(rng)` / `set_state!(rng, s) -> rng`	save and restore the position only
`copy(rng)`, `show(io, rng)`	independent copy; one-line summary, or the three anchors when displayed at the REPL
the full `Random` API	`rand`, `rand!`, `randn`, `randexp`, `shuffle`, ranges, every integer and float type; `randn` and `randexp` by inversion

On the generator object

Call	Meaning
`next_stream!(gen) -> rng`	the next non-overlapping stream
`next_stream!(gen, n) -> Vector`	the next n of them; the thread-safe way to obtain streams for parallel work
`srand!(gen, seed) -> gen`	reset the seed the next `next_stream!` will use
`get_state(gen)` / `set_state!(gen, s) -> gen`	save and restore that seed
`show(io, gen)`	display it, on one line or in full as above

What is *not* common

Name	Where	Why not everywhere
`short_jump!(rng)`	xoshiro/xoroshiro, PCG	the family's spelling of a jump by exactly the substream distance; use `next_substream!` in portable code
`long_jump!(rng)`	xoshiro/xoroshiro, PCG	jumps by the stream distance in place. Meaningless for a counter-based generator, where changing stream means changing *key*, which belongs to the generator object
`checkseed`, `checkseed63`	MRG32k3a and MRG63k3a, xoshiro/xoroshiro	those two families have invalid seeds — the MRG moduli with no all-zero component, and the xoshiro all-zero state, which is a fixed point. PCG and the counter-based families accept every value
`stream_key`	counter-based	the key schedule hook; there is nothing to schedule in a recurrence-based generator
`next(rng)` *(internal)*	all	raw word output, before any conversion

Signature	Description
`MRG32k3a()`	default seed `[12345, 12345, 12345, 12345, 12345, 12345]`
`MRG32k3a(seed::Integer)`	integer seed, folded into the valid seed space
`MRG32k3a(x::Vector{Int})`	seed all three states (`Cg = Bg = Ig = x`)
`MRG32k3a(x, y, z::Vector{Int})`	explicit current/substream/stream starts

4.3 MRG32k3a

Constructors

Seeds are validated with `checkseed`; invalid seeds throw an `AssertionError`.

Functions

`rand(rng::MRG32k3a) -> Float64` Uniform in (0, 1) with ~32 bits of resolution. This is the native output.

`rand(rng::MRG32k3a, T)` Supported T: `Float64`, `Float32`, `Float16`, `UInt8…UInt128`, `Int8…Int128`, `Bool`, and any `UnitRange` (e.g. `rand(rng, 1:10)`).

`reset_stream!(rng) -> rng` Set the current state (`Cg`) *and* the substream start (`Bg`) back to the stream start (`Ig`).

`reset_substream!(rng) -> rng` Set `Cg` back to `Bg`.

`next_substream!(rng) -> rng` Advance `Bg` to the next substream boundary (jump of ≈ 2^76 steps) and set `Cg = Bg`.

`advance_state!(rng, e::Integer, c::Integer) -> rng` Jump forward/backward inside the current substream by `n` steps where `n = 2^e + c`. Negative `e` or `c` move backwards. Cost is O(max(|e|, log |c|)) matrix operations.

`get_state(rng) -> Vector{Int}` Return a copy of `Cg`.

`checkseed(x) -> Bool` `true` iff `x` has length 6, all entries ≥ 0, entries 1–3 are < m1 and not all zero, entries 4–6 are < m2 and not all zero.

`DEFAULT_SEED` The constant seed vector used by `MRG32k3a()` and `MRG32k3aGen()`.

`copy(m::MRG32k3a) -> MRG32k3a` Independent deep copy of the generator (all three state vectors).

Stream generation

Signature	Description
`MRG32k3aGen()`, `MRG32k3aGen(12345)`, `MRG32k3aGen(v)`	create a stream generator
`next_stream!(gen) -> MRG32k3a`	return a new stream; internal seed leaps 2^127 values ahead
`srand!(gen, seed) -> gen`	reset the stored seed
`get_state(gen) -> Vector{Int}` / `set_state!(gen, s)`	save and restore it

4.4 MRG63k3a

The same combined MRG in 64-bit arithmetic: L'Ecuyer (1999), Table II, fourth entry. Two moduli just under 2^63 (`m1 = 2^63 - 6645`, `m2 = 2^63 - 21129`), period ≈ 2^377, and just under 63 random bits per step against 32 for MRG32k3a. Everything below mirrors the MRG32k3a section; only the numbers differ.

Constructors

Signature	Description
`MRG63k3a()`	default seed `[12345, 12345, 12345, 12345, 12345, 12345]`
`MRG63k3a(seed::Integer)`	integer seed, folded into the valid seed space
`MRG63k3a(x::Vector{Int})`	seed all three states (`Cg = Bg = Ig = x`)
`MRG63k3a(x, y, z::Vector{Int})`	explicit current/substream/stream starts

Seeds are validated with `checkseed63`; invalid seeds throw an `AssertionError`.

Functions

`rand(rng::MRG63k3a) -> Float64` Uniform in (0, 1), from an integer with ~63 bits of resolution, of which a `Float64` keeps 53. This is the native output, and it reproduces L'Ecuyer's C implementation value for value.

`rand(rng::MRG63k3a, T)` Supported `T`: as for MRG32k3a. Words are assembled from 32-bit chunks, so a `UInt64` costs two steps (four for MRG32k3a) and a `UInt32` one.

`next_substream!(rng) -> rng` Advance `Bg` by `2^150` steps and set `Cg = Bg`.

`reset_stream!`, **`reset_substream!`**, **`advance_state!`**, **`get_state`**, **`set_state!`**, **`srand!`**, **`copy`** Exactly as for MRG32k3a, with one internal difference that does not show: MRG63k3a keeps its state one step ahead of the position (Vigna's third optimization, worth 7% here), so `get_state` returns the seed representation by stepping the stored vector back, and the constructors, `set_state!` and `Random.seed!` step a seed forward. Reading `rng.Cg` directly is therefore not the same thing as `get_state(rng)` for this generator. See the Implementation Notes.

`checkseed63(x) -> Bool` `true` iff `x` has length 6, all entries ≥ 0, entries 1–3 are < m1 and not all zero, entries 4–6 are < m2 and not all zero.

`DEFAULT_SEED63` The constant seed vector used by `MRG63k3a()` and `MRG63k3aGen()`.

Stream generation

Signature	Description
`MRG63k3aGen()`, `MRG63k3aGen(12345)`, `MRG63k3aGen(v)`	create a stream generator
`next_stream!(gen) -> MRG63k3a`	return a new stream; internal seed leaps `2^250` values ahead
`srand!(gen, seed) -> gen`	reset the stored seed
`get_state(gen) -> Vector{Int}` / `set_state!(gen, s)`	save and restore it

The jumps are unaffected by that shift: the jump matrices commute with the one-step matrix, so a jumped shifted state is the shifted jumped state.

The stream and substream distances are not L'Ecuyer's: he published jump matrices for MRG32k3a only. `2^250` and `2^150` are this package's choice, scaled from his `2^127` and `2^76` by the ratio of the two periods, and the matrices are computed at precompilation rather than tabulated.

4.5 Xoshiro256+

Constructors

Signature	Description
`Xoshiro256p()`	the package default seed, 12345, through splitmix64
`Xoshiro256p(seed::Integer)`	integer seed, through splitmix64 (never all-zero)
`Xoshiro256p(s::NTuple{4,UInt64})` / `Xoshiro256p(v::Vector{<:Unsigned})`	`Cg = Bg = Ig = s`
`Xoshiro256p(x, y, z)`	explicit current/substream/stream states

Functions

`rand(rng::Xoshiro256p) -> Float64` Uniform in (0, 1), from the top 52 bits of one 64-bit output.

`rand(rng::Xoshiro256p, T)` Supported T: `Float64`, `Float32`, `Float16`, and the integer and `Bool` types of the `Random` API. Ranges (`rand(rng, 1:10)`) go through the standard `Random` sampler, which rejects rather than folding and is therefore unbiased.

`srand!(rng, seed::Vector{UInt64}) -> rng` Re-seed an existing generator (first 4 words are used).

`reset_stream!(rng) -> rng`, **`reset_substream!(rng) -> rng`**, **`next_substream!(rng) -> rng`** Same semantics as for MRG32k3a; `next_substream!` performs a `short_jump!` (leap of ≈ `2^96` values).

`short_jump!(rng) -> rng` Jump to the beginning of the next `2^96`-value block (Vigna's short jump).

`long_jump!(rng) -> rng` Leap ≈ `2^192` values ahead — used to delimit streams.

`get_state(rng) -> Vector{UInt64}` Copy of the current state `Cg`.

`copy(rng) -> Xoshiro256p` Independent copy.

(internal) **`next(rng) -> UInt64`**: raw 64-bit output; also available as `rand(rng, UInt64)`.

Stream generation

Signature	Description
`Xoshiro256plusGen()`, `Xoshiro256plusGen(12345)`, `Xoshiro256plusGen(v)`	create a stream generator (a seed *vector* must have exactly 4 words)
`next_stream!(gen) -> Xoshiro256p`	new independent stream (`long_jump!` from the stored seed)
`srand!(gen, seed::Vector{UInt64}) -> gen`	reset the stored seed
`get_state(gen) -> Vector{UInt64}`	seed that will be used by the next `next_stream!` call

4.6 Xoshiro / xoroshiro families

All variants share one implementation (`LinRNG{N,S}` internally, `N` = state words, `S` = scrambler) and expose the same API as Xoshiro256+ above. Exported types:

RNGs	Stream generators
`Xoroshiro128p`, `Xoroshiro128ss`, `Xoroshiro128pp`	`Xoroshiro128pGen`, `Xoroshiro128ssGen`, `Xoroshiro128ppGen`
`Xoshiro256p`, `Xoshiro256ss`, `Xoshiro256pp`	`Xoshiro256plusGen` (also `Xoshiro256pGen`), `Xoshiro256ssGen`, `Xoshiro256ppGen`
`Xoshiro512p`, `Xoshiro512ss`, `Xoshiro512pp`	`Xoshiro512pGen`, `Xoshiro512ssGen`, `Xoshiro512ppGen`

Constructors accept an `NTuple{N,UInt64}` or a `Vector{<:Unsigned}` of length `N` (1 or 3 arguments). Available functions: `rand` (Float64/Float32/Float16, UInt64, ranges), `srand!`, `reset_stream!`, `reset_substream!`, `next_substream!`, `short_jump!`, `long_jump!`, `get_state`, `copy`.

`advance_state!(rng, e::Integer, c::Integer) -> rng` Jump forward by `n` steps (`n = 2^e + c` if `e > 0`, `n = -2^(-e) + c` if `e < 0`, `n = c` if `e = 0`) or backward if `n < 0`. Implemented by computing the jump polynomial `x^n mod p(x)` over GF(2) (`p` = the family's characteristic polynomial) and applying it to the current state; distances are reduced modulo the period. Only `Cg` moves — stream/substream boundaries are kept. Cost is $O((64N)^2)$ GF(2) operations (≈ 10–500 ms depending on family); for the standard distances prefer `short_jump!`/`long_jump!`, which use precomputed constants and are allocation-free.

Jump semantics (all xoshiro-family generators):

- `short_jump!(rng)` — jump anchored at the current **substream start** (`Bg`); lands at the next substream boundary (`2^64` values for xoroshiro128, `2^128` for xoshiro256, `2^256` for xoshiro512).
- `long_jump!(rng)` — jump anchored at the **stream start** (`Ig`); delimits independent streams.
- `next_stream!(gen)` applies the long-jump polynomial to the stored seed.

All jump constants are the official ones from xoshiro.di.unimi.it and outputs are validated byte-for-byte against the original C implementations.

4.7 Where the families actually differ

Every call in The common interface is available on all seventeen generators, so there is no availability matrix to consult. What differs is cost and representation:

For the standard distances on a xoshiro or PCG generator prefer `short_jump!`/`long_jump!` over `advance_state!`: they use precomputed constants (xoshiro) or a single doubling chain (PCG) and are allocation-free.

4.8 PCG

PCG (O'Neill 2014) runs a 128-bit linear congruential recurrence and permutes the state into 64 bits of output. `PCG64` is the default bit generator of NumPy, and the outputs here match it exactly for the same state; `PCG64DXSM` is the variant NumPy recommends for large-scale parallel work.

	MRG32k3a	MRG63k3a	xoshiro / xoroshiro	PCG64, PCG64DXSM	Philox, Threefry
Native output	`Float64`, ~32-bit resolution	`Float64`, ~63-bit resolution	`UInt64`	`UInt64`	a block of four words
`get_state` returns	`Vector{Int}` (6)	`Vector{Int}` (6)	`Vector{UInt64}` (N)	`UInt128`	`(ctr, key, buffer, idx)`
Cost of `advance_state!`	O(e) matrix products	O(e) matrix products	O((64N)²) GF(2) ops, ~10–500 ms	O(log n) multiplies	O(1), a counter addition
`short_jump!`, `long_jump!`	—	—	yes	yes	—
Rejected seeds	checkseed: two moduli, no all-zero component	checkseed63: same, wider moduli	checkseed: the all-zero state	none	none
Whole-block `rand!`	—	—	—	—	yes, ~1.7× the scalar path
Extra hooks	checkseed, `DEFAULT_SEED`	checkseed63, `DEFAULT_SEED63`	jump polynomials	—	`stream_key`

Streams come from the closed-form LCG jump, not from PCG's own increment-based stream mechanism, which this package deliberately does not expose — see the Implementation Notes for the reason and the FAQ for what to do instead when reproducing a NumPy pipeline.

`RandomDataStreams.PCG64` – Type.

```
PCG64 <: AbstractStreamableRNG
```

PCG64 (O'Neill 2014) with the XSL-RR output permutation: a 128-bit linear congruential state, period 2^128, 64 bits of output per draw. This is the default bit generator of NumPy, and the outputs here match it exactly for the same state.

Streams are 2^96 values apart and substreams 2^64, giving 2^32 streams of 2^32 substreams — the same scale as `Xoroshiro128p`, so it suits mild parallelism rather than large simulations. Jumping to an arbitrary position costs `O(log n)` multiplications, the cheapest of any generator in the package.

The increment is fixed: PCG's own increment-based "streams" are not used here, because two increments can define orbits that differ by a constant offset. See the implementation notes.

source

`RandomDataStreams.PCG64DXSM` – Type.

```
PCG64DXSM <: AbstractStreamableRNG
```

PCG64 with the DXSM output permutation and the cheap 64-bit multiplier, the variant NumPy recommends for large-scale parallel use. Same state size, period and stream structure as PCG64, and the outputs match NumPy's `PCG64DXSM` exactly for the same state.

source

`RandomDataStreams.PCG64Gen` – Type.

```
PCG64Gen([seed])
```

Hands out independent PCG64 streams, 2^32 * (2^64 + 1) + 1 values apart.

source

RandomDataStreams.PCG64DXSMGen – Type.

```
PCG64DXSMGen([seed])
```

Hands out independent PCG64DXSM streams, 2^32 * (2^64 + 1) + 1 values apart.

source

RandomDataStreams.PCGRNG – Type.

```
PCGRNG{V}
```

Permuted congruential generator on 128 bits of state, where V names the output permutation. Use the aliases PCG64 and PCG64DXSM.

Like the other recurrence-based generators of the package it carries three checkpoints: the current state Cg, the start of the current substream Bg and the start of the current stream Ig.

source

RandomDataStreams.PCGGen – Type.

```
PCGGen{V} <: AbstractRNGStream
```

Hands out non-overlapping PCGRNG streams: each call to next_stream! applies the long jump to the stored seed. Use the aliases PCG64Gen and PCG64DXSMGen.

source

4.9 Counter-based generators

A counter-based RNG produces its output block as a keyed bijection of a counter, R = b_K(C), rather than by iterating a state transition. CBRNG holds the machinery every such family shares — counter, key, block buffer, streams and substreams — so a variant only supplies its bijection.

RandomDataStreams.CBRNG – Type.

```
CBRNG{B,W,N,K}
```

Counter-based generator, where B names the bijection, W is the word type, N the number of words in an output block and K the number of key words. Concrete aliases (PhiloxRNG, Philox4x64RNG, ...) are what user code normally names.

Streams are indexed by the key, substreams by the high half of the counter; see next_stream! and next_substream!.

source

RandomDataStreams.stream_key – Function.

```
stream_key(::Val{B}, seed) -> key
```

Key schedule of the family `B`: the bijection mapping the seed of a stream — the position `0, 1, 2, ...` walked by `CBGen`, or a value the user supplied through `srand!` — to the key the bijection is actually keyed with.

The default is the identity, which is what Philox uses: L'Ecuyer et al. (2021, Sec. 3) report that consecutive small keys are safe there, on the strength of the statistical testing of Salmon et al. (2011) and the ten rounds of encoding.

A family whose bijection is weak for structured keys **must** override this method with a schedule that hashes the seed. The paper's worked example is Squares (Widynski 2020): key `0` makes the output identically zero, and a small key `k` makes roughly the first `2^16 / k` outputs zero, so handing out the seeds `1, 2, ..., s` as keys "will be very bad unless the keys are hashed to random-looking values by another mechanism".

source

Philox

`RandomDataStreams.PhiloxGen` – Type.

```
PhiloxGen([key])
```

Hands out independent `PhiloxRNG` streams, one distinct key each.

source

`RandomDataStreams.PhiloxRNG` – Type.

```
PhiloxRNG([key, [ctr]])
```

Philox4x32-10 counter-based stream. The key identifies the stream, the 128-bit counter its position; sub-streams are 2^64 blocks (2^66 draws) apart.

source

`RandomDataStreams.Philox4x64Gen` – Type.

```
Philox4x64Gen([key])
```

Hands out independent `Philox4x64RNG` streams, one distinct key each.

source

`RandomDataStreams.Philox4x64RNG` – Type.

```
Philox4x64RNG([key, [ctr]])
```

Philox4x64-10 counter-based stream: same construction as `PhiloxRNG` with 64-bit words, so a block holds four `UInt64` and `rand(rng, UInt64)` costs no bit assembly. Only the low 128 bits of its 256-bit counter space are used.

source

Threefry

Same construction as Philox, with no multiplication at all: every round is add / rotate / xor, which makes Threefry reproducible bit-for-bit across architectures, including ones with no fast integer multiply. Salmon et al. (2011) found it the fastest of the family on the CPUs of the time and recommend twenty rounds there; on a current x86 our own measurement puts Philox4x64-10 ahead (see the performance notes).

`RandomDataStreams.Threefry4x64Gen` – Type.

```
Threefry4x64Gen([seed])
```

Hands out independent `Threefry4x64RNG` streams, one distinct key each.

source

RandomDataStreams.Threefry4x64RNG – Type.

```
Threefry4x64RNG([key, [ctr]])
```

Threefry4x64-20 counter-based stream, the variant Salmon et al. recommend on CPUs. Four 64-bit key words identify the stream; substreams are 2^64 blocks apart. Only the low 128 bits of its 256-bit counter space are used, which still gives 2^130 draws per substream.

The round count is a parameter of the bijection rather than of the type, so a faster 13-round variant — enough to pass Crush, per Salmon et al. — is three lines:

```
RandomDataStreams.bijection(::Val{:threefry4x64_13}, ctr::NTuple{4,UInt64},
↪  key::NTuple{4,UInt64}) =
    RandomDataStreams.threefry(ctr, key, Val(13))
RandomDataStreams._variant_name(::Type{<:RandomDataStreams.CBRNG{:threefry4x64_13}}) =
↪  "Threefry4x64-13"
const Threefry4x64_13RNG = RandomDataStreams.CBRNG{:threefry4x64_13,UInt64,4,4}
```

source

RandomDataStreams.Threefry4x32Gen – Type.

```
Threefry4x32Gen([seed])
```

Hands out independent `Threefry4x32RNG` streams, one distinct key each.

source

RandomDataStreams.Threefry4x32RNG – Type.

```
Threefry4x32RNG([key, [ctr]])
```

Threefry4x32-20 counter-based stream: same construction as `Threefry4x64RNG` with 32-bit words, for a 128-bit counter and a 128-bit key.

source

Part V

Docstrings

Chapter 5

Docstrings

Auto-generated documentation from the package docstrings.

5.1 Abstract types

`RandomDataStreams.AbstractRNGStream` – Type.

> `AbstractRNGStream` is an abstraction for creating RNG objects with non-overlaping random numbers.
>
> source

`RandomDataStreams.AbstractStreamableRNG` – Type.

> `AbstractStreamableRNG` is a special kind of RNG where one can move between different stream of number that are known to be non-overlapping.
>
> Every concrete subtype implements the same interface, so that code written against `AbstractStreamableRNG` works with any generator of the package:
>
> | operation | meaning |
> |---:|---:|
> | `next_substream!(rng)` | move to the start of the next substream |
> | `reset_substream!(rng)` | rewind to the start of the current substream |
> | `reset_stream!(rng)` | rewind to the start of the current stream |
> | `advance_state!(rng, e, c)` | move by `n` draws, `n` given by `(e, c)`; `n < 0` moves backwards |
> | `get_state(rng)` | current position, in a generator-specific representation |
> | `set_state!(rng, state)` | restore a position obtained from `get_state` |
> | `srand!(rng, seed)` | reseed, resetting the stream and substream boundaries |
>
> The unit of `advance_state!` is one `rand(rng)` draw, for every generator, so that a jump written against `AbstractStreamableRNG` means the same thing everywhere. `get_state` and `set_state!` are inverses — `set_state!(rng, get_state(rng))` leaves `rng` unchanged — and they move only the current position, never the stream or substream boundaries. The representation returned by `get_state` differs between families, so generic code should treat it as opaque and only ever hand it back to `set_state!`.
>
> Streams themselves are produced by an `AbstractRNGStream` object, which implements the same operations on its side:
>
> An integer seed is expanded through splitmix64 and folded into whatever the family accepts, so `MRG32k3aGen(12345)`, `Xoshiro256ppGen(12345)`, `PhiloxGen(12345)` and `PCG64Gen(12345)` all mean the same kind of thing.

operation	meaning
`Gen()`	a generator seeded with the package default, 12345
`Gen(seed)`	seeded with an integer, or with the family's own seed vector
`next_stream!(gen)`	the next non-overlapping stream
`next_stream!(gen, n)`	the next n of them, as a vector
`srand!(gen, seed)`	reset the seed the next `next_stream!` will use
`get_state(gen)` / `set_state!(gen, s)`	save and restore that seed

A generator object is a factory, and it is **not thread-safe**: it rewrites the seed of the next stream on every call, so concurrent calls to `next_stream!` hand out overlapping streams, silently. Take the streams first, with `next_stream!(gen, n)`, then parallelise over them; streams themselves share no state and need no synchronisation.

Not part of this contract: `short_jump!` and `long_jump!`, which are the xoshiro and PCG spelling of a jump by the substream and stream distance. Use `next_substream!` for portable code. `long_jump!` has no meaning for a counter-based generator at all, where moving to another stream means taking another key, an operation that belongs to the generator object.

source

5.2 MRG32k3a

`RandomDataStreams.MRG32k3a` – Type.

`MRG32k3a()` will generate an instance of `MRG32k3a` with initial seeds `DEFAULT_SEED = [ 12345, 12345, 12345, 12345, 12345, 12345 ]`

`MRG32k3a(x::Vector{Int})` will generate an instance of `MRG32k3a` with initial seeds `x`

`MRG32k3a(x::Vector{Int}, y::Vector{Int}, z::Vector{Int})` will generate an instance of `MRG32k3a` with `Cg = x`, `Bg = y` and `Ig = z`

Examples

```
julia> rng = MRG32k3a();

julia> rand(rng)
0.12701112204657714
```

source

`RandomDataStreams.MRG32k3aGen` – Type.

An object that generates independent random number streams.

source

`RandomDataStreams.checkseed` – Function.

```
checkseed(s::NTuple{N,UInt64}) -> Bool
checkseed(v::AbstractVector{<:Unsigned}) -> Bool
```

`true` unless the words are all zero. The all-zero state maps to itself under every xoshiro/xoroshiro transition, so it is the one state the families forbid; there is no other constraint. (The `Vector{Int}` method is MRG32k3a's, whose seed space is constrained differently.)

source

5.3 MRG63k3a

RandomDataStreams.MRG63k3a – Type.

MRG63k3a() will generate an instance of MRG63k3a with initial seeds DEFAULT_SEED63 = [12345, 12345, 12345, 12345, 12345, 12345]

MRG63k3a(x::Vector{Int}) will generate an instance of MRG63k3a with initial seeds x

MRG63k3a(x::Vector{Int}, y::Vector{Int}, z::Vector{Int}) will generate an instance of MRG63k3a positioned at x, with the substream starting at y and the stream at z. All three are seeds, in the representation get_state returns; the state actually stored runs one step ahead of them (see below).

MRG63k3a is the 64-bit-arithmetic member of L'Ecuyer's (1999) combined MRG family: same structure as MRG32k3a, moduli just under 2^63 instead of 2^32, period ≈ 2^377 instead of 2^191. Its step yields just under 63 random bits, against just under 32 for MRG32k3a, so a machine word costs two steps here and four there.

The state is stored one step ahead of the position, so that a draw returns a pair the previous draw already computed and the processor overlaps the output with the next state. That is internal: seeds, get_state, set_state! and show all speak the same representation as MRG32k3a.

Examples

```
julia> rng = MRG63k3a();

julia> rand(rng)
0.999964376179128
```

source

RandomDataStreams.MRG63k3aGen – Type.

An object that generates independent MRG63k3a streams, 2^250 numbers apart.

source

RandomDataStreams.checkseed63 – Function.

```
checkseed63(x::Vector{Int}) -> Bool
```

True when x is a valid MRG63k3a seed: six non-negative integers, the first three below m1 = 2^63 - 6645 and not all zero, the last three below m2 = 2^63 - 21129 and not all zero.

source

5.4 Xoshiro / xoroshiro families

RandomDataStreams.LinRNG – Type.

```
LinRNG{N,S} <: AbstractStreamableRNG
```

Generic xoshiro/xoroshiro generator: state of N 64-bit words and scrambler S ∈ (:plus, :starstar, :plusplus). Holds three checkpoints — current state (Cg), substream start (Bg) and stream start (Ig) — enabling reset_stream!, reset_substream! and anchored jumps. Use the exported aliases (Xoroshiro128p, ..., Xoshiro512pp) instead.

source

RandomDataStreams.Xoroshiro128p – Type.

```
Xoroshiro128p <: AbstractStreamableRNG
```

xoroshiro128+ (Blackman & Vigna): 128-bit state, period 2^128 - 1, output `s0 + s1`. Fastest small-state generator for floating-point use; the four lower bits have low linear complexity and it has a very mild Hamming-weight dependency after ~5 TB of output. Suitable only for mild parallelism (2^32 streams x 2^64 substreams).

source

RandomDataStreams.Xoroshiro128ss – Type.

```
Xoroshiro128ss <: AbstractStreamableRNG
```

xoroshiro128** (Blackman & Vigna): 128-bit state, period 2^128 - 1, all-purpose scrambler (`rotl(s1*5,7)*9`), 1-dimensionally equidistributed. Suitable only for mild parallelism (2^32 streams x 2^64 substreams).

source

RandomDataStreams.Xoroshiro128pp – Type.

```
Xoroshiro128pp <: AbstractStreamableRNG
```

xoroshiro128++ (Blackman & Vigna): 128-bit state, period 2^128 - 1, all-purpose scrambler (`rotl(s0+s1,17)+s0`). Same parallelism limits as the other 128-bit variants.

source

RandomDataStreams.Xoshiro256p – Type.

```
Xoshiro256p <: AbstractStreamableRNG
```

xoshiro256+ (Blackman & Vigna): 256-bit state, period 2^256 - 1, output `s0 + s3`. Slightly faster than **/++ when generating only floating-point numbers from the upper bits; avoid for raw 64-bit consumption.

source

RandomDataStreams.Xoshiro256ss – Type.

```
Xoshiro256ss <: AbstractStreamableRNG
```

xoshiro256** (Blackman & Vigna): 256-bit state, period 2^256 - 1, all-purpose, 3-dimensionally equidistributed (default PRNG of Lua and .NET 6).

source

RandomDataStreams.Xoshiro256pp – Type.

```
Xoshiro256pp <: AbstractStreamableRNG
```

xoshiro256++ (Blackman & Vigna): 256-bit state, period 2^256 - 1, all-purpose, 3-dimensionally equidistributed (default PRNG of Julia and Rust's SmallRng). The recommended general-purpose variant.

Examples

```
julia> rng = Xoshiro256pp(fill(UInt64(42), 4));

julia> rand(rng, UInt64)
0x000000002a00002a
```

source

RandomDataStreams.Xoshiro512p – Type.

```
Xoshiro512p <: AbstractStreamableRNG
```

xoshiro512+ (Blackman & Vigna): 512-bit state, period 2^512 - 1, output `s0 + s2`. Floating-point oriented; same caveats as `Xoshiro256p`.

source

RandomDataStreams.Xoshiro512ss – Type.

```
Xoshiro512ss <: AbstractStreamableRNG
```

xoshiro512** (Blackman & Vigna): 512-bit state, period 2^512 - 1, all-purpose, 7-dimensionally equidistributed.

source

RandomDataStreams.Xoshiro512pp – Type.

```
Xoshiro512pp <: AbstractStreamableRNG
```

xoshiro512++ (Blackman & Vigna): 512-bit state, period 2^512 - 1, all-purpose. Offers 2^256 substreams of 2^256 values — more than any application needs; prefer `Xoshiro256pp` unless you have a specific reason.

source

Stream generators

RandomDataStreams.LinGen – Type.

```
LinGen{N,S} <: AbstractRNGStream
```

Stream generator producing non-overlapping streams of `LinRNG{N,S}`: each call to `next_stream!` applies the family's long jump to the stored seed. Use the exported aliases (`Xoroshiro128pGen`, ..., `Xoshiro512ppGen`) instead.

source

RandomDataStreams.Xoroshiro128pGen – Type.

```
Xoroshiro128pGen <: AbstractRNGStream
```

Stream generator minting non-overlapping `Xoroshiro128p` streams via a long jump (2^96 values) per call. Seeds are 2 `UInt64` words.

source

RandomDataStreams.Xoroshiro128ssGen – Type.

```
Xoroshiro128ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping `Xoroshiro128ss` streams via a long jump (2^96 values) per call. Seeds are 2 `UInt64` words.

source

RandomDataStreams.Xoroshiro128ppGen – Type.

```
Xoroshiro128ppGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoroshiro128pp streams via a long jump (2^96 values) per call. Seeds are 2 UInt64 words.

source

RandomDataStreams.Xoshiro256plusGen – Type.

```
Xoshiro256plusGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256p streams via a long jump (2^192 values) per call. Seeds are 4 UInt64 words.

source

RandomDataStreams.Xoshiro256pGen – Type.

```
Xoshiro256pGen <: AbstractRNGStream
```

Regular spelling of Xoshiro256plusGen, matching Xoshiro256ssGen, Xoshiro512pGen and the rest. The two names are the same type.

source

RandomDataStreams.Xoshiro256ssGen – Type.

```
Xoshiro256ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256ss streams via a long jump (2^192 values) per call. Seeds are 4 UInt64 words.

source

RandomDataStreams.Xoshiro256ppGen – Type.

```
Xoshiro256ppGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256pp streams via a long jump (2^192 values) per call. Seeds are 4 UInt64 words.

source

RandomDataStreams.Xoshiro512pGen – Type.

```
Xoshiro512pGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro512p streams via a long jump (2^384 values) per call. Seeds are 8 UInt64 words.

source

RandomDataStreams.Xoshiro512ssGen – Type.

```
Xoshiro512ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping `Xoshiro512ss` streams via a long jump (2^384 values) per call. Seeds are 8 `UInt64` words.

source

`RandomDataStreams.Xoshiro512ppGen` – Type.

```
Xoshiro512ppGen <: AbstractRNGStream
```

Stream generator minting non-overlapping `Xoshiro512pp` streams via a long jump (2^384 values) per call. Seeds are 8 `UInt64` words.

source

5.5 Counter-based generators

`RandomDataStreams.CBGen` – Type.

```
CBGen{B,W,N,K}
```

Hands out independent `CBRNG` streams, each with its own key. Concrete aliases (`PhiloxGen`, `Philox4x64Gen`, ...) are what user code normally names.

The generator walks an odometer over the *seeds* `0, 1, 2, ...` and maps each one through the family's key schedule `stream_key` to obtain the actual key; see that function for why the two are not the same thing.

source

GPU primitives

`RandomDataStreams.philox4x32_10` – Function.

```
philox4x32_10(ctr::NTuple{4,UInt32}, key::NTuple{2,UInt32}) -> NTuple{4,UInt32}
```

Ten-round Philox4x32 bijection, the primitive `PhiloxRNG` is built on. Exposed directly for GPU kernels: every thread calls this with its own counter and the stream's key to get its block, with no shared state and no synchronization between threads.

source

`RandomDataStreams.philox4x32_counter` – Function.

```
philox4x32_counter(hi::UInt64, lo::UInt64) -> NTuple{4,UInt32}
```

Pack a 128-bit Philox4x32 counter out of two 64-bit halves: `hi` typically addresses a substream – a GPU thread's global index, so distinct threads never collide – and `lo` the position within it – a per-thread draw index, so one thread can take several draws. This is the same high/low split `next_substream!` uses on a `PhiloxRNG`, exposed as a plain function because a kernel has no stream object to call it on.

Built out of `UInt64` shifts and truncations rather than a `UInt128` intermediate, since not every GPU compiler backend lowers 128-bit integer arithmetic; only bitwise ops on 64-bit words are needed here, and every backend handles those natively.

source

`RandomDataStreams.open01` – Function.

```
open01(u::UInt64) -> Float64
open01(u::UInt32) -> Float32
```

Map a raw generator word to a uniform value in `(0, 1)`: the conversion every floating-point draw of a counter-based generator applies, scalar or array. Exposed so that a kernel producing raw words – the package's CUDA extension, or one of your own built on `philox4x32_10` – matches the CPU bit for bit. A `UInt64` word gives a `Float64` from its top 52 bits, a `UInt32` word a `Float32` from its top 23, each as an odd multiple of `2^-53` (`2^-24`). The one floating-point operation is a subtraction whose exact result is representable, so every IEEE 754 device computes the same value.

source

5.6 Normal and exponential variates

`RandomDataStreams.norminv` – Function.

```
norminv(u) -> z
```

The standard normal quantile function Φ^{-1}: the `z` with $\Phi(z) = u$, for `u` in `(0, 1)`, and `-Inf`, `Inf` at 0 and 1. It is the map every generator of the package uses for `randn` – `randn(rng)` is `norminv(rand(rng, Float64))` – and is exported so that a randomized quasi-Monte Carlo point, or the uniforms of a kernel of one's own, can be mapped the same way.

Wichura's AS 241 (relative error about 1e-16), computed with correctly rounded IEEE 754 operations only, so a GPU gives the same bits as the CPU. Odd about 1/2: `norminv(1 - u) == -norminv(u)` whenever `1 - u` is exact, which it is for every uniform the word generators return.

A `Float32` argument is computed in single precision, by AS 241's PPND7 (about 1e-7 relative error), with the same portability: that is what `randn32` uses. A `Float16` argument goes through the `Float32` method. For the `Float64` quantile of a narrower uniform, convert it first.

source

`Base.randn` – Method.

```
randn(rng::AbstractStreamableRNG) -> Float64
```

A standard normal by inversion: `norminv(rand(rng, Float64))`, one uniform per variate. Every generator of the package draws its normals this way, rather than by the standard library's Ziggurat, so that antithetic variates, common random numbers and randomized quasi-Monte Carlo carry through from the uniforms to the normals; see "Normal variates and variance reduction" in the manual. The CUDA extension's `randn!` computes the same values.

source

`Random.randexp` – Method.

```
randexp(rng::AbstractStreamableRNG) -> Float64
```

An exponential variate of rate 1 by inversion, `-log(1 - rand(rng, Float64))`, for the same reasons as `randn`.

source

`RandomDataStreams.randn32` – Function.

```
randn32(rng) -> Float32
randn32!(rng, A::AbstractArray{Float32}) -> A
```

Single-precision standard normals by inversion: `norminv(rand(rng, Float32))`, computed in `Float32` throughout. Faster than `randn(rng, Float32)` on a GPU whose `Float64` rate is low – the CUDA extension's `randn32!` gives the same values on a `CuArray` – but its tails end at ±5.29, where `randn(rng, Float32)`, the `Float64` normal rounded, reaches ±8.21. One `rand(rng, Float32)` per normal: one 32-bit word for the counter-based generators, one step for the others.

source

`RandomDataStreams.randn32!` – Function.

```
randn32!(rng, A::AbstractArray{Float32}) -> A
```

Fill `A` with `randn32` draws, in order: `norminv` of the `Float32` uniforms `rand!(rng, A)` would give. On a `CuArray`, the CUDA extension gives the same values.

source

The GPU fills below take their methods from the CUDA extension, loaded with CUDA.jl. `randn!` and `randexp!` on a `CuArray` need no name of their own: they are the standard ones, and give the CPU's values.

`RandomDataStreams.randn_boxmuller!` – Function.

```
randn_boxmuller!(rng, A) -> A
```

Standard-normal fill via Box-Muller on the GPU, two uniforms of one Philox4x32 block to two normals. Faster than `randn!` and branch-free, but it does not carry antithetic variates or randomized quasi-Monte Carlo through, and it matches no CPU output. Only defined for `PhiloxRNG`/`CuArray`, by the CUDA extension – see its docstring.

source

`RandomDataStreams.randn_polar!` – Function.

```
randn_polar!(rng, A) -> A
```

Standard-normal fill via Marsaglia's polar method (accept-reject), kept only for comparison against inversion (`randn!`) and `randn_boxmuller!`. Only defined for `PhiloxRNG/CuArray{Float64}`, by the CUDA extension – see its docstring.

source

`RandomDataStreams.randn_inversion!` – Function.

```
randn_inversion!(rng, A) -> A
```

Deprecated: `randn!` draws by inversion now, on the CPU and on the GPU, with the same values on both. Forwards to it, on a `CuArray`, when the CUDA extension is loaded.

source

Functions

`Base.rand` – Method.

Produces a raw random number with 32 bits of precision, in (0, 1): the combination k lies in `1:m1` and is scaled by `1/(m1 + 1)`.

source

`Base.rand` – Method.

Produces a raw random number with 63 bits of precision, of which a Float64 keeps 53, in (0, 1). The combination k lies in `1:m1` and is scaled by `1/(m1 + 1)`, but for the twelve largest values of k the product rounds to exactly 1.0 – in the reference C code as well – so the result is capped at `prevfloat(1.0)`. Every other draw is the reference value.

source

`Base.rand` – Method.

Return a random Float64 in (0, 1): the top 52 bits of one output, as an odd multiple of 2^-53. The top bits are the ones the + scramblers leave strongest.

source

`Base.rand` – Method.

Generates a `Float32` in (0, 1) from any xoshiro/xoroshiro generator: the top 23 bits of one output, as an odd multiple of 2^-24.

source

`Base.rand` – Method.

Generates a `Float16` in (0, 1) from any xoshiro/xoroshiro generator: the top 10 bits of one output, as an odd multiple of 2^-11.

source

`Base.rand` – Method.

Return a random Float64 in (0, 1): the top 52 bits of the 64-bit output, as an odd multiple of 2^-53.

source

`Base.rand` – Method.

Generates a `Float32` in (0, 1) from any counter-based generator: the top 23 bits of the draw `rand(rng, UInt32)` would return, as an odd multiple of 2^-24. A family with 32-bit words therefore spends one word per `Float32`, not the two a `Float64` takes.

source

`Base.rand` – Method.

Generates a `Float16` in (0, 1) from any counter-based generator: the top 10 bits of the draw `rand(rng, UInt32)` would return, as an odd multiple of 2^-11.

source

`Base.rand` – Method.

Return a random Float64 in (0, 1): the top 52 bits of one output, as an odd multiple of 2^-53.

source

`Base.rand` – Method.

Generates a `Float32` in (0, 1) from a PCG generator: the top 23 bits of one output, as an odd multiple of 2^-24.

source

`Base.rand` – Method.

Generates a `Float16` in (0, 1) from a PCG generator: the top 10 bits of one output, as an odd multiple of 2^-11.

source

`Base.rand` – Method.

Hook required by Random's generic machinery for Float64-native generators: produces a Float64 in [1, 2). Everything else (ints, arrays, shuffle...) derives from this without further plumbing.

source

`Base.rand` – Method.

Hook required by Random's generic machinery for Float64-native generators: produces a Float64 in [1, 2). Everything else (ints, arrays, shuffle...) derives from this without further plumbing.

source

`RandomDataStreams.short_jump!` – Function.

Jumps forward by 2^96 values (xoroshiro128), 2^128 (xoshiro256) or 2^256 (xoshiro512): the start of the next non-overlapping substream.

source

Jumps forward by 2^64 + 1 values: the start of the next non-overlapping substream. The distance is odd on purpose; see the implementation notes.

source

`RandomDataStreams.long_jump!` – Function.

Jumps forward by 2^96 (xoroshiro128), 2^192 (xoshiro256) or 2^384 (xoshiro512) values: the start of the next independent stream.

source

Jumps forward by 2^32 * (2^64 + 1) + 1 values: the start of the next independent stream. The distance is odd on purpose; see the implementation notes.

source

`RandomDataStreams.advance_state!` – Function.

Given a random number generator, jumps n steps forward if n > 0 (or -n steps backwards if n < 0), where

if e > 0, let n = 2^e + c; if e < 0, let n = -2^(-e) + c; if e = 0, let n = c.

Examples

```
julia> rng = MRG32k3a();

julia> advance_state!(rng, Int64(2), Int64(-1));   # skip n = 2^2 - 1 = 3 values

julia> rand(rng)
0.8258468629271136
```

source

Given a random number generator, jumps n steps forward if n > 0 (or -n steps backwards if n < 0), where

if e > 0, let n = 2^e + c; if e < 0, let n = -2^(-e) + c; if e = 0, let n = c.

Examples

```
julia> rng = MRG63k3a();

julia> advance_state!(rng, Int64(2), Int64(-1));   # skip n = 2^2 - 1 = 3 values

julia> rand(rng)
0.8707612110911583
```

source

Given a random number generator, jumps n steps forward if n > 0 (or -n steps backwards if n < 0), where

if e > 0, let n = 2^e + c; if e < 0, let n = -2^(-e) + c; if e = 0, let n = c.

The distance is reduced modulo the period (2^(64N) - 1), so any magnitude is accepted. Only the current position moves; the stream/substream boundaries (`Bg`, `Ig`) are left untouched. Costs O((64N)^2) GF(2) operations.

source

Given a random number generator, jumps n steps forward if n > 0 (or -n steps backwards if n < 0), where

if e > 0, let n = 2^e + c; if e < 0, let n = -2^(-e) + c; if e = 0, let n = c.

The distance is reduced modulo the period 2^128, so any magnitude is accepted and a backward jump costs the same as a forward one. Only the current position moves; the stream and substream boundaries are left untouched.

source

Given a random number generator, jumps n words forward if n > 0 (or -n words backwards if n < 0), where

if e > 0, let n = 2^e + c; if e < 0, let n = -2^(-e) + c; if e = 0, let n = c.

The unit is one `rand(rng)` draw, the same as for every other generator of the package. A draw takes 64 bits, so it consumes two words of a 32-bit family and one word of a 64-bit one. The distance is reduced modulo the counter space, so any magnitude is accepted. The stream and substream boundaries are not affected, only the current position.

source

`RandomDataStreams.srand!` – Function.

```
srand!(gen::MRG32k3aGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

source

```
srand!(rng::MRG32k3a, seed) -> rng
```

Reseeds the generator, resetting the stream and substream boundaries to `seed`.

source

```
srand!(gen::MRG63k3aGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

source

```
srand!(rng::MRG63k3a, seed) -> rng
```

Reseeds the generator, resetting the stream and substream boundaries to `seed`.

source

Seeds a generator with the first words of `seed`.

source

```
srand!(gen, seed::Vector{UInt64}) -> gen
```

Resets the seed that the next `next_stream!` call will use.

source

Seeds a generator, resetting the stream and substream boundaries to the seed.

source

```
srand!(gen::PCGGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

source

```
srand!(rng::CBRNG, key) -> rng
```

Reseeds the stream with `key` (a tuple or a vector of key words) and rewinds its counter to the start of the stream.

source

```
srand!(gen::CBGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

source

`RandomDataStreams.get_state` – Function.

`get_state` return the seed used to generate non-overlaping MRGs

source

```
get_state(rng::MRG32k3a) -> Vector{Int}
```

Copy of the current state (`Cg`); restore it with `set_state!`.

source

```
get_state(gen::MRG63k3aGen) -> Vector{Int}
```

The seed the next `next_stream!` will use; restore it with `set_state!`.

source

```
get_state(rng::MRG63k3a) -> Vector{Int}
```

The current position, in the same representation as a seed; restore it with `set_state!`. The internal vector runs one step ahead of it, so this steps back by one – a matrix-vector product, off any hot path.

source

```
get_state(rng::LinRNG) -> Vector{UInt64}
```

Copy of the current state (`Cg`); restore it into a fresh generator via its three-argument constructor.

source

```
get_state(gen) -> Vector{UInt64}
```

Seed that will be used by the next `next_stream!` call.

source

```
get_state(rng::PCGRNG) -> UInt128
```

Current state. Hand it back to `set_state!` to return here.

source

```
get_state(gen::PCGGen) -> UInt128
```

Seed that will be used by the next `next_stream!` call.

source

```
get_state(rng::CBRNG) -> (ctr, key, buffer, idx)
```

Full internal state: the index of the next block to produce, the key, the block most recently produced and the position of the next word to consume in it.

source

```
get_state(gen::CBGen) -> NTuple{K,W}
```

Seed that the next `next_stream!` call will use. This is the position in the key schedule, not the key itself; the two coincide only when `stream_key` is the identity.

source

`RandomDataStreams.set_state!` – Function.

```
set_state!(gen::MRG32k3aGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

source

```
set_state!(rng::MRG32k3a, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current state moves; the stream and substream boundaries (`Bg`, `Ig`) are untouched.

source

```
set_state!(gen::MRG63k3aGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

source

```
set_state!(rng::MRG63k3a, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current state moves; the stream and substream boundaries (`Bg`, `Ig`) are untouched.

source

```
set_state!(rng::LinRNG, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current position moves; the stream and substream boundaries (`Bg`, `Ig`) are untouched.

source

```
set_state!(gen::LinGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

source

```
set_state!(rng::PCGRNG, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current position moves; the stream and substream boundaries are untouched.

source

```
set_state!(gen::PCGGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

source

```
set_state!(rng::CBRNG, state) -> rng
```

Restores the current position of the stream from a `get_state(rng)` value: the counter, the key, the buffered block and the index within it.

source

```
set_state!(gen::CBGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

source

RandomDataStreams.next_stream! – Function.

```
next_stream!(gen, n::Integer) -> Vector
```

Return n successive non-overlapping streams from gen, in order.

This is the thread-safe way to obtain streams for parallel work. A generator object holds the seed of the next stream and rewrites it on every call, so calling `next_stream!(gen)` from several threads at once is a data race: the threads read the same seed and hand out streams that overlap. The failure is silent — the result is not an error but a set of correlated streams, which is exactly the guarantee the package exists to provide.

Call this once, from one thread, and give each replication its own element:

```
using Base.Threads

gen  = MRG32k3aGen()
rngs = next_stream!(gen, nrep)          # serial, here

@threads for r in 1:nrep
    rng = rngs[r]                       # replication r owns stream r
    ...
end
```

Streams themselves carry no shared state, so drawing from different streams on different threads needs no synchronisation. Indexing them by replication rather than by thread makes the results independent of the number of threads.

source

Given an RNG generator object, returns the next RNG stream.

source

Given an RNG generator object, returns the next RNG stream.

source

Given an RNG generator object, returns the next RNG stream.

source

Given an RNG generator object, returns the next RNG stream.

source

```
next_stream!(gen) -> rng
```

Returns the next independent stream and moves the generator on to the following seed. Seeds are walked in lexicographic order and mapped through `stream_key`, a bijection, so distinct streams never share a key and their counter spaces cannot overlap.

source

RandomDataStreams.reset_stream! – Function.

Resets a given random number generator to the beginning of the current stream.

source

Resets a given random number generator to the beginning of the current stream.

source

```
reset_stream!(rng) -> rng
```

Rewind the generator to the very beginning of its current stream (`Cg = Bg = Ig`).

source

```
reset_stream!(rng) -> rng
```

Rewind the generator to the very beginning of its current stream.

source

```
reset_stream!(rng) -> rng
```

Rewind the generator to the very beginning of its current stream (counter 0).

source

`RandomDataStreams.reset_substream!` – Function.

Resets a given random number generator to the beginning of the current substream.

source

Resets a given random number generator to the beginning of the current substream.

source

```
reset_substream!(rng) -> rng
```

Rewind the generator to the beginning of its current substream (`Cg = Bg`).

source

```
reset_substream!(rng) -> rng
```

Rewind the generator to the beginning of its current substream.

source

```
reset_substream!(rng) -> rng
```

Rewind the generator to the beginning of its current substream.

source

`RandomDataStreams.next_substream!` – Function.

Takes a random number generator and shifts seed to next substream.

source

Takes a random number generator and shifts seed to next substream, `2^150` numbers ahead.

source

```
next_substream!(rng) -> rng
```

Move the generator to the start of the next substream (anchored jump from Bg): 2^64 values for xoroshiro128, 2^128 for xoshiro256, 2^256 for xoshiro512.

source

```
next_substream!(rng) -> rng
```

Move the generator to the start of the next substream: an anchored jump of 2^64 + 1 values from the current substream boundary.

source

```
next_substream!(rng) -> rng
```

Move the generator to the start of the next substream: a jump of 2^(_substream_shift) blocks in the counter (2^64 blocks for Philox4x32-10).

source

Part VI

Implementation Notes

Chapter 6

Implementation Notes

6.1 MRG32k3a

MRG32k3a is L'Ecuyer's combined multiple recursive generator of order 3. It maintains two 3-component integer states updated by the recurrences

```
p1 = (1403580 · x2 − 810728 · x1) mod m1,      m1 = 2^32 − 209
p2 = (527612  · y2 − 1370589 · y0) mod m2,    m2 = 2^32 − 22853
u  = (p1 − p2) / (m1 + 1)   (with wrap-around) ∈ (0, 1)
```

The combined generator has period ≈ 2^191.

Modular arithmetic without overflow, and without tests

The recurrence follows Vigna's testless formulation (https://github.com/vigna/MRG32k3a). The two negative coefficients are carried as positive numbers and subtracted, and a multiple of the modulus is added, so the argument of `%` can never be negative and the residual needs no correction afterwards; the combination `z = p1 - p2 mod m1` is done with an arithmetic shift rather than a comparison. Worth about 15% here — 204 million `Float64` per second before, 230 to 235 after depending on the run — and the stream is unchanged bit for bit.

It is worth 15% and not the factor of two Vigna reports, because most of what he removes by hand LLVM already removes for us: the previous formulation, with its two residual corrections and its comparison, compiled to 72 instructions with **no branch and no division** — the corrections had become conditional moves and the constant moduli multiply-shift sequences. What is left to gain is the fix-up a *signed* remainder needs, which disappears once the dividend is known non-negative: 72 instructions become 68.

Two variants measured and rejected: keeping the state in six scalar fields instead of a `Vector{Int64}` is *slower* (226 against 235) while breaking the constructors and `Cg`; and computing the output from the current state before the update, which needs the state kept one step ahead to preserve the stream, gives 231 and changes what `get_state` and the jump matrices operate on.

That second variant is Vigna's third optimization, and the conclusion does **not** carry over to MRG63k3a, where it is worth 7% and is used — see below. Re-measured here with the same harness, MRG32k3a gives 236 against 235: the 32-bit step is short enough that the combination is already hidden behind it, so there is nothing to overlap.

Modular arithmetic in the jump machinery

All state components stay below `m1 < 2^32`, so products fit in `Float64` exactly (< 2^53). The helper `MultModM(a, s, c, m)` computes `(a·s + c) mod m` using this fact and falls back to a split-by-2^17 computation when intermediate values would exceed 2^53. Matrix helpers:

- `MatVecModM(A, s, m)` — matrix–vector product mod m
- `MatMatModM(A, B, m)` — matrix product mod m
- `MatTwoPowModM(A, e, m)` — A raised to the power 2^e
- `MatPowModM(A, n, m)` — A^n by binary exponentiation

Streams, substreams, jumps

Moving a stream forward by k steps is a linear operation on its state, hence a precomputed matrix power applied to the state vector:

Operation	Matrix	Step
`next_stream!(gen)`	`A1p127`, `A2p127`	2^127 values
`next_substream!(rng)`	`A1p76`, `A2p76`	2^76 values
`advance_state!(rng, e, c)`	`MatTwoPowModM` / `InvA1`, `InvA2`	arbitrary (inverse matrices allow backward jumps)

The RNG keeps three checkpoints: `Cg` (current), `Bg` (substream start), `Ig` (stream start), which is what makes `reset_substream!` and `reset_stream!` O(1).

6.2 MRG63k3a

The same construction as MRG32k3a, sized for 64-bit arithmetic: the fourth entry of Table II in L'Ecuyer (1999), which he implements in C as `MRG63k3a`.

```
p1 = (1754669720 · x2 −  3182104042 · x0) mod m1,   m1 = 2^63 − 6645
p2 = (31387477935 · y2 − 6199136374 · y0) mod m2,   m2 = 2^63 − 21129
u  = (p1 − p2) / (m1 + 1)   (with wrap-around) ∈ (0, 1)
```

Period ≈ 2^377, against 2^191; entropy per step `log2(m1) = 63.0` bits, against 32.0. The package reproduces L'Ecuyer's C implementation value for value, checked on three seeds and a million draws — with one exception, the twelve draws that round to exactly 1.0 (see *Floating-point outputs* below).

Reduction without a 128-bit division

The products no longer fit in a `Float64` mantissa — that is the whole reason this generator is a separate implementation and not a parameter set — so they are computed with `widemul` into an `Int128`. What must be avoided then is the remainder: a 128-bit `%` is a software routine, and it would dominate the step.

Both moduli are pseudo-Mersenne, `m = 2^63 − c` with c small, so the reduction is two shifts, a multiply and an add. Splitting the 128-bit value at bit 63 as `t = hi·2^63 + lo` gives

```
t ≡ hi·c + lo   (mod m),         since 2^63 ≡ c (mod m).
```

With the same testless formulation as MRG32k3a — the negative coefficient carried positive and a multiple of the modulus added — `t` stays below 2^99, so `hi < 2^36` and `hi·c < 2^51`: the folded value is below 2^63 + 2^51, which is under 2m, and one conditional subtraction finishes it. The combination `z = p1 − p2 mod m1` then uses the same arithmetic shift as MRG32k3a. The whole step compiles to 61 instructions with no division and no branch — the two conditional subtractions become conditional moves.

The reference C code takes a different route to the same values, Bratley, Fox & Schrage's approximate factoring (`q = m ÷ a`, `r = m mod a`), which keeps every intermediate inside a 64-bit signed integer at the cost of two divisions and two branches per coefficient. That is the right choice for a 1999 C compiler and the wrong one here: the reduction above is what makes MRG63k3a *faster* than MRG32k3a per random bit rather than slower. The alternative was measured, not assumed — a plain 128-bit remainder instead of the fold runs at 19 million draws per second against 196, a factor of ten.

The state runs one step ahead

Vigna's three optimizations for MRG32k3a are the testless formulation above, the branchless combination, and computing the output from the *current* state so that the processor overlaps it with the next state. The first two are shared with MRG32k3a; the third is used **here only**, because it is the one place where the two generators measure differently:

	MRG32k3a	MRG63k3a
output from the new state	236 M/s	183 M/s
output from the current state	235 M/s	**196 M/s**

The 63-bit step is a `widemul` and a fold, long enough that the combination sitting at the end of it is visible; the 32-bit step is a `Float64` multiply and a compiler-optimised `%`, short enough that it is not. So `next_pair!` returns the pair already in `Cg[3]`, `Cg[6]` and computes the next one, and the state vector is one step ahead of the position.

That shift is invisible from outside. It costs nothing anywhere it might have: the jump matrices commute with the one-step matrix, so `advance_state!`, `next_substream!`, `next_stream!` and the `Bg`/`Ig` checkpoints operate on the shifted vector unchanged, and jumping a shifted state gives the shifted jumped state. Only the boundary converts — the constructors and `Random.seed!` step a seed forward, `get_state` and `show` step the stored vector back — at one matrix-vector product each, on paths that are never hot. The stream is identical bit for bit, which is what the reference vectors from L'Ecuyer's C code check.

Jump matrices

`MultModM` here is the straightforward `widemul` plus 128-bit `mod`, since it serves only the jump machinery, never the draw. The matrix helpers are the same four as for MRG32k3a.

The jump distances are **not** L'Ecuyer's: he published `A1p76`/`A1p127` for MRG32k3a and nothing for MRG63k3a. This package uses `2^150` between substreams and `2^250` between streams — his `2^76` and `2^127` scaled by the ratio of the two periods, `377/191 ≈ 1.97` — which cuts the period into `2^127` streams of `2^100` substreams of `2^150` numbers each. The four matrices are computed by repeated squaring at precompilation rather than tabulated, roughly 400 matrix products, and the test suite checks the arithmetic that produces them: `A · InvA = I` for both components, `(A^(2^150))^(2^100) = A^(2^250)`, `next_substream!` equal to `advance_state!(rng, 150, 0)`, and `next_stream!` equal to `advance_state!(rng, 250, 0)`. The formulas behind `InvA1` and `InvA2` are checked by a route that does not depend on this generator at all: applied to MRG32k3a's coefficients they must reproduce the inverse matrices L'Ecuyer tabulates, and they do.

Integer outputs

Same construction as MRG32k3a, one size up: `z ∈ [1, m1]` is truncated to 32-bit chunks and wider words are concatenations of consecutive chunks. A `UInt64` is two steps, against four; a `UInt32` is one, against two. Because `m1 = 2^63 − 6645`, a residue class modulo 2^32 holds 2^31 values give or take one, so the departure from uniformity is of order `2^-31` per chunk against `2^-16` for MRG32k3a. Still not exact — no exact uniform word comes out of a non-power-of-two modulus without rejection — but two orders of magnitude closer, at half the cost.

`Float64` remains the native output (`Random.rng_native_52`): one step covers a double, where declaring the word native would make every double cost two.

The one-step-ahead ordering helps the paths whose critical path ends in the combination — `Float64` and `UInt32` — and not `UInt64`, where two steps run back to back and the recurrence, not the combination, is the chain.

6.3 Xoshiro256+

xoshiro256+ (Blackman & Vigna, 2019) keeps four 64-bit words and produces one word per step with only shifts, XORs and rotations:

```
result = s1 + s4
t = s2 << 17
s3 = xor(s3, s1);  s4 = xor(s4, s2);  s2 = xor(s2, s3);  s1 = xor(s1, s4);  s3 = xor(s3, t);  s4 =
↪  rotl(s4, 45)
```

The output word is the sum of two internal words (+ variant): fastest of the xoshiro family for float generation, though the low three bits of the raw output have limited linear complexity — irrelevant once scaled to `Float64`, which uses the top 52 bits. Period: `2^256 − 1`.

Jumps

`short_jump!` and `long_jump!` use Vigna's published jump polynomials. They are computed as XOR-linear combinations of states visited while stepping through the 256 bits of each jump constant. RandomDataStreams.jl hoists the jump constants into compile-time tuples and evaluates jumps allocation-free on immutable state tuples.

RandomDataStreams semantics differ subtly from the reference C code: jumps are **anchored** at stream boundaries. `short_jump!(rng)` first resets `Cg ← Bg`, then applies the jump polynomial, and stores the result in both `Cg` and `Bg`; `long_jump!(rng)` does the same with respect to `Ig`. This matches L'Ecuyer's stream model (`next_substream!`/`reset_stream!`) and makes scenario replay reproducible regardless of consumption inside the current substream.

Families and scramblers

The package implements all 64-bit variants from xoshiro.di.unimi.it through a single generic type `LinRNG{N,S}` (state words × scrambler):

- transition `_lin_step` for `NTuple{2}` (xoroshiro128: `a=24, b=16, c=37`), `NTuple{4}` (xoshiro256) and `NTuple{8}` (xoshiro512);
- scramblers `+`, `**` (`rotl(s2·5,7)·9`) and `++` (`rotl(s1+s4,23)+s1` for 256; family-specific constants);
- per-family jump constants shared by all scramblers.

Exported aliases: `Xoroshiro128p/ss/pp`, `Xoshiro256p/ss/pp`, `Xoshiro512p/ss/pp` plus matching `*Gen` stream generators.

All nine variants are regression-tested against byte-exact outputs produced by the original C implementations compiled with gcc (sequences, short jumps and long jumps).

Arbitrary forward/backward jumps

Because the transition is multiplication by x in GF(2)[x]/p(x) — with p the family's characteristic polynomial, as published in Vigna's reference files — one can jump any distance n in constant time by applying the polynomial `x^n mod p` through the same accumulate-and-step loop as the fixed jumps. Backward distances use the multiplicative order of x: `x^(-n) = x^(2^deg-1-n)` (deg = 64N). RandomDataStreams.jl implements a small GF(2) polynomial engine (`_poly_mul_mod`, `_poly_pow_x`, BigInt exponents reduced modulo the period) and exposes it as `advance_state!(rng, e, c)` with the exact distance convention of MRG32k3a. Correctness is property-tested: fixed jumps coincide with `short_jump!`, round trips (+k then -k) restore the state bit-for-bit, and backward-jumped generators re-emit previously seen values.

State representation

The state lives in `NTuple{4,UInt64}` fields. Immutable tuples let the JIT keep the whole working set in registers inside `next`, eliminating bounds checks and heap traffic that a `Vector{UInt64}` representation incurs. Measured throughput: ≈ 10^9 draws/s on a single core.

6.4 PCG

`PCGRNG{V}` runs a linear congruential recurrence on 128 bits, `s <- a*s + c (mod 2^128)`, and permutes the state into 64 bits of output. Two variants ship: `PCG64` with the XSL-RR permutation, which is what `numpy.random.default_rng()` returns, and `PCG64DXSM` with the DXSM permutation and the cheap 64-bit multiplier, which NumPy recommends for large-scale parallel work. They differ in one more detail that matters for bit-exactness: PCG64 steps the state and permutes the result, DXSM permutes the state and then steps it. Both are pinned by known-answer vectors taken from NumPy.

Jumps are closed-form. For an LCG,

```
s_n = a^n * s_0 + c * (a^n - 1)/(a - 1)   (mod 2^128),
```

which Brown's (1994) doubling recurrence evaluates in `O(log n)` multiplications. `advance_state!` therefore costs the same at any distance, and a backward jump of n is the forward jump of `2^128 - n`, so it costs the same as a forward one. This is the cheapest arbitrary jump in the package: the xoshiro families pay `O(deg^2)` GF(2) operations for the same operation, tens to hundreds of milliseconds, and MRG32k3a a sequence of matrix products. **The jump distances are odd, and that is not cosmetic.** The obvious choice — `2^64` between substreams, `2^96` between streams, mirroring the xoroshiro128 layout — is wrong for a linear congruential recurrence. By the lifting-the-exponent lemma,

```
v2(a^n - 1) = v2(a - 1) + v2(a + 1) + v2(n) - 1 = 2 + v2(n)   (n even),
```

so a jump of `2^m` produces a jump multiplier congruent to 1 modulo `2^(m+2)`. With `m = 96`, the states of successive streams agree in their low 98 bits up to an arithmetic progression. Each stream still passes SmallCrush on its own; interleaving 64 of them round-robin fails it outright, with 14 of 15 p-values at 0. The package's inter-stream battery found this, which is the argument for having one.

An odd distance has `v2(n) = 0`, leaving the jump multiplier as far from the identity as the multiplier itself. Substreams are therefore `2^64 + 1` apart and streams `2^32 * (2^64 + 1) + 1`, which is still `2^32` streams of `2^32` substreams of `2^64` draws and still non-overlapping — the last substream of a stream ends two values below the next stream's start. With those distances the interleaved battery passes. Both the parity of the distances and the 2-adic distance of the resulting jump multipliers are asserted in the test suite.

The increment is fixed, and that is deliberate. PCG has its own notion of a stream: every odd increment `c` gives a full-period orbit, and NumPy exposes it as part of the state. Those orbits are not known to be independent. Writing `t_n = s_n + h` and matching the two recurrences gives

```
h * (a - 1) = c1 - c2,
```

which has a solution whenever 4 divides `c1 - c2` — the full-period condition forces `a = 1 (mod 4)`, and `v2(a - 1) = 2` for the PCG multipliers. For half of all pairs of odd increments, then, the two "independent streams" are the same state sequence translated by a constant. Taking `c2 = c1 - (a - 1)` makes that constant 1, and the two output sequences then differ by a mean Hamming distance of 2 bits out of 64, against 32 for unrelated sequences, with 94% of draws differing in at most 4 bits.

That is an adversarial pair; most pairs of increments are far less structured and would show nothing. The objection is not that every pair is bad but that no criterion is published for telling the good pairs from the bad ones, which is exactly what L'Ecuyer et al. (2021, Sec. 3) argue against — the same objection they raise against Squares. So the package fixes the increment to the reference value for every PCG stream and derives streams from jumps, whose non-overlap is a statement about distances along one orbit. The algebra above is locked in the test suite.

6.5 Counter-based generators

`CBRNG{B,W,N,K}` holds what every counter-based family shares — the counter, the key, the block most recently produced and the index of the next word in it. This is the state `(k, i, j)` of L'Ecuyer et al. (2021, Sec. 3), with `f(k, i, j) = (k, i + I[j = d-1], (j + 1) mod d)` and `d = N`. A variant only supplies `bijection(::Val{B}, ctr, key)`.

Two families are built on it. **Philox** keeps both halves of a fixed-constant multiplication and xors the high halves into the other words, with a Weyl sequence for the round key. **Threefry** is a reduction of the Threefish cipher used in Skein and contains no multiplication at all — add, rotate, xor only — so it is reproducible bit-for-bit on any architecture, including ones with no fast integer multiply. Adding it required only its bijection and its aliases, no stream machinery.

Its rounds are unrolled at compile time with a generated function. Each round uses a different pair of Skein rotation constants, so a plain loop indexes the constant table with a value LLVM cannot fold; unrolling is worth a factor of 5.8 on the bijection (7.7 → 44.8 Mblocks/s), which is the difference between Threefry being unusable and being in the same class as Philox. The round count is a parameter of the bijection, not of the type, so the faster 13-round variant is three lines of user code.

Streams are keys. **Substreams** partition the counter, following the same paper: "one can use the `c0 < c` most significant bits of the counter to determine the substream, and the remaining `c1 = c - c0` bits for the position within the substream". `_substream_shift` splits the counter in half, so Philox4x32-10 has `2^64` substreams of `2^64` blocks each.

Key schedules. `CBGen` walks the seeds `0, 1, 2, ...` and maps each through `stream_key`, a bijection that defaults to the identity. Consecutive small keys are safe for Philox — the ten rounds of encoding absorb the structure, as reported by Salmon et al. (2011) and discussed in L'Ecuyer et al. (2021, Sec. 3) — but they are *not* safe for every counter-based construction, so a family whose bijection is weak for structured keys must override `stream_key` with a schedule that hashes the seed.

6.6 Integer outputs of MRG32k3a

MRG32k3a returns `z = (x1 - x2) mod m1` with `m1 = 2^32 - 209`, prime, scaled to `u = z / (m1 + 1)`. Its native output is a uniform integer over a range that is **not** a power of two, carrying `log2(m1) = 32.0` bits. This is why the reference literature — L'Ecuyer (1999), and the RNGStreams package of L'Ecuyer, Simard, Chen & Kelton (2002) whose stream API this package follows — exposes only `RandU01()` and `RandInt(i, j) = i + floor((j - i + 1) * RandU01())`, and no native-word or raw-bit primitive: no exact uniform word comes out of one draw without rejection.

Machine integers are therefore an extension beyond what that literature specifies and validates, and they are built here by assembling 16-bit chunks of `z`, one MRG step per chunk — four steps for a `UInt64`. Truncating `z` to 16 bits carries a relative bias of `2^-16`, since `2^16` does not divide `m1`; taking the low bits is sound because the modulus is prime, so there is no low-bit weakness of the kind a power-of-two LCG has.

The cheaper alternative — assembling a word from the mantissas of two `[1, 2)` draws, two steps instead of four — was implemented here and removed. It assumes 52 random bits per double, and MRG32k3a supplies 32; the low mantissa bits are a near-deterministic function of the high ones. The result passed every `U(0,1)` battery and failed SmallCrush on its own bit stream with six p-values at zero, bit 12 coming out set in two draws out of three. `test/test_bits.jl` is the regression net.

Narrower types are truncations of one chunk. None of these integers is exactly uniform over its full width — fine for indexing, shuffling and flags in a simulation, not for cryptographic use. `MRG63k3a` above is the same construction with twice the chunk width and half the steps per word; it is the one to reach for when a run consumes integers rather than floats.

6.7 Floating-point outputs

Every generator returns floating-point draws in the open interval `(0, 1)`, in `Float64`, `Float32` and `Float16`, through `rand(rng)`, `rand(rng, T)`, `rand(rng, T, dims...)` and `rand!` alike. Neither end is reachable, so inversion — a quantile function, `-log(u)`, Φ^{-1}`(u)` — is finite for every draw.

For the families whose native output is a word (xoshiro/xoroshiro, PCG, counter-based), the top `p − 1` bits of the word, read as an integer `k`, give an odd multiple of `2^-p`, where `p` is the precision of the type:

```
Float64   u = (2k + 1) · 2^-53,   k = top 52 bits,   u ∈ [2^-53, 1 − 2^-53]
Float32   u = (2k + 1) · 2^-24,   k = top 23 bits,   u ∈ [2^-24, 1 − 2^-24]
Float16   u = (2k + 1) · 2^-11,   k = top 10 bits,   u ∈ [2^-11, 1 − 2^-11]
```

It is computed by filling the mantissa of 1.0 with `k` and subtracting the float just below 1, `(1 + k·2^(1−p)) − (1 − 2^-p)`: the exact result is representable, so the subtraction is exact on any IEEE 754 device, which is what lets a GPU kernel reproduce the CPU's draws bit for bit (see Streams & Substreams).

Every value is exact, and the draws are the midpoints of `2^(p−1)` equal cells, so `1 − u` is exact and is itself a possible draw: antithetic pairs stay on the same grid. The open interval costs one bit of resolution. A `Float32` takes one 64-bit output on the xoshiro and PCG families, and on the counter-based ones the 32-bit draw `rand(rng, UInt32)` returns — so Philox4x32 spends one word per `Float32`, not two.

The constructions this replaced each failed at one end. Dividing by `typemax(UInt64)` (xoshiro) returned exactly 1 for the top `2^10` outputs, and 0 for an output of 0. Rounding a `Float64` draw to `Float32` returned `1.0f0` once in `2^25` draws, once in `2^12` for `Float16`. Filling the mantissa of 1.0 and subtracting 1 — PCG, the counter-based families, and the standard library's `Float32` path, which array fills took — returned 0.

The MRG families produce a `Float64` in `(0, 1)` natively, `k / (m1 + 1)` with `k ∈ [1, m1]`, and every path now returns that value. Array fills used to go through `Random`'s `(1 + u) − 1` instead, which rounds `u` to a multiple of `2^-52` and, for MRG63k3a, returned 0 below `2^-53` and 1 above `1 − 2^-53`. A `Float32` or `Float16` takes the top 32 bits of the fraction through the construction above, one step per draw.

MRG63k3a needs one more guard. For the twelve largest values of `k`, `k / (m1 + 1)` lies within `2^-54` of 1 and the product rounds to exactly 1.0 — in L'Ecuyer's C code as well. Those draws are capped at `prevfloat(1.0)`, the only place where the package departs from the reference values.

6.8 Normal and exponential variates

`randn(rng)` is Φ^{-1} of one uniform and `randexp(rng)` is `-log(1 - u)` of one; Streams & Substreams explains why. The two maps live in `src/normal.jl` and are built to give the same bits on every IEEE 754 device.

Φ^{-1} is Wichura's AS 241, PPND16 (*Applied Statistics* 37, 1988): rational functions of degree 7 in three regions. The central one, $|u - 1/2| \le 0.425$, covers 85% of the draws and needs no logarithm; the two tails take $r = \sqrt{-\log \min(u, 1-u)}$, split at $r = 5$. Against 256-bit MPFR its relative error stayed below 5×10^{-16}, 4 ulp at worst over 20 000 points spread across the three regions. The logarithm is fdlibm's `e_log.c`, within 0.81 ulp in the same check.

Both use only operations IEEE 754 rounds correctly — +, −, ×, ÷, `sqrt`, `fma` — and integer operations on the bits. No libm: Julia's `log` and CUDA's libdevice `log` differ in the last bits. And no product is ever an operand of a sum, because the GPU compiler contracts `a * b + c` into an fma where the CPU's does not: before every such sum was written as the fma itself, one exponential in 200 000 came out an ulp apart on the device. The test suite now compares the maps on the device and on the host over the extremes of the uniform grid, both tails, the boundaries between the regions and a million further points, and the device fills against the CPU fills for the four counter-based families.

The antithetic identity `norminv(1 - u) == -norminv(u)` is exact on the word generators' grid of odd multiples of 2^{-53}: `1 - u` and `u - 1/2` are exact there, and the rational function is computed from $|u - 1/2|$ and its sign. A `Float32` or `Float16` normal from `randn` is the `Float64` one rounded, as in the standard library.

`randn32` and `norminv(::Float32)` are the same construction in single precision: AS 241's PPND7, polynomials of degree 3 over 2 in the same three regions, and FreeBSD's `e_logf.c`, the `Float32` conversion of fdlibm's logarithm, with the same fma rule. Both were checked against the `Float64` versions over every `Float32` they accept: the logarithm within 0.854 ulp over all positive finite `Float32`, Φ^{-1} within 5.4 `Float32` ulps over all of $(0, 1)$ — the worst near 1.5×10^{-11}, in a tail no uniform of the 2^{-24} grid reaches — and the antithetic identity exact, and the map monotone, over all 2^{23} points of that grid. The device gives the same bits over all of them.

6.9 Testing strategy

Reference sequences were captured from the reference implementations (L'Ecuyer's C code for MRG32k3a and MRG63k3a — for the latter, ten doubles, the draw one million steps later, and the raw combined integer on three seeds; Vigna's constants for the xoshiro jumps) and regression-checked after optimization: identical outputs are produced for every supported type, plus reset/jump/state round-trips.

6.10 Performance snapshot

Reproduce with `julia -O3 scripts/benchmarks/throughput.jl`, which prints the Julia version and CPU it ran on. On a hybrid CPU (Intel 12th generation and later), pin the process first — `taskset -c 0 julia -O3 …` — and say which core type was used: otherwise the scheduler moves the run between performance and efficiency cores and the minimum reflects whichever was fastest. Figures below are one run on a Skylake x86_64, Julia 1.12.5, single core; treat them as ratios and re-run on the machine that matters to you.

The measurement method is part of the result. Naive loops disagree by nearly a factor of two, so the script fixes one method and states it: BenchmarkTools with the minimum sample; scalar draws accumulated with `xor` on the raw bits, because a floating-point + chain has four cycles of latency — longer than a draw from the fastest generators here, so it would measure the adder; nothing written to memory in the scalar loop, because storing every draw turns the benchmark into one of memory bandwidth (the same xoshiro reads 867 M/s with an accumulator and 481 M/s storing into an 8 MB vector); and `rand!` measured into a 4096-element vector, small enough to stay in cache.

Scalar draws: millions per second, and nanoseconds per draw. Every generator the package ships is listed, each scrambler variant separately. Run-to-run variation is a few percent, so read differences below that as noise.

Array fill with `rand!`, millions of elements per second (nanoseconds per element is `1000` over the figure):

Generator	Float64	ns	UInt64	ns	UInt32	ns
MRG32k3a	236	4.25	47	21.06	95	10.48
MRG63k3a	196	5.10	93	10.76	199	5.04
Xoroshiro128p	697	1.43	941	1.06	1090	0.92
Xoroshiro128ss	655	1.53	876	1.14	848	1.18
Xoroshiro128pp	599	1.67	818	1.22	926	1.08
Xoshiro256p	788	1.27	1242	0.81	1221	0.82
Xoshiro256ss	715	1.40	1140	0.88	962	1.04
Xoshiro256pp	688	1.45	1032	0.97	1055	0.95
Xoshiro512p	570	1.75	798	1.25	742	1.35
Xoshiro512ss	523	1.91	742	1.35	614	1.63
Xoshiro512pp	532	1.88	725	1.38	665	1.50
PCG64	504	1.99	599	1.67	605	1.65
PCG64DXSM	561	1.78	743	1.35	745	1.34
Philox4x32-10	93	10.75	109	9.20	272	3.68
Philox4x64-10	203	4.94	262	3.82	259	3.86
Threefry4x32-20	84	11.86	92	10.89	186	5.39
Threefry4x64-20	147	6.80	153	6.52	157	6.35

Generator	Float64	UInt64	UInt32
MRG32k3a	163	47	94
MRG63k3a	184	93	185
Xoroshiro128p	546	581	581
Xoroshiro128ss	531	538	520
Xoroshiro128pp	507	500	511
Xoshiro256p	501	519	520
Xoshiro256ss	492	520	472
Xoshiro256pp	471	505	507
Xoshiro512p	380	391	426
Xoshiro512ss	375	393	390
Xoshiro512pp	377	389	408
PCG64	575	643	628
PCG64DXSM	632	744	735
Philox4x32-10	158	178	349
Philox4x64-10	359	339	257
Threefry4x32-20	100	106	206
Threefry4x64-20	209	181	156

What the tables say. Within each xoshiro family the ordering is the same and the spread is small: + is fastest, then **, then ++, within about 15% of each other — the scrambler is a couple of instructions on top of a shared transition. Across families, state size costs: 128 and 256 bits are close, 512 bits is a third slower. The counter-based generators sit an order of magnitude below on `Float64`, which is what buying a keyed bijection per block costs, and the two 32-bit ciphers are fastest in `UInt32`, where a draw is one cipher word rather than two.

The two PCG variants land between the xoshiro families and the MRG generators, at roughly two thirds of `Xoshiro256p`: a 128-bit multiply per draw is more work than a handful of shifts and xors. `PCG64DXSM` is the faster of the two despite its extra output multiply, because its "cheap" 64-bit multiplier turns the 128x128 state multiplication into a 64x128 one. Both are the fastest generators in the package under `rand!`, ahead of every xoshiro: the whole state is one 128-bit word, so the bulk loop touches far less memory per element than a four- or eight-word state does.

The one outlier is `MRG32k3a` in `UInt64`, at 21.1 ns against 4.3 ns for its `Float64`: a 64-bit word costs four MRG steps, because the modulus is not a power of two and the word is assembled from 16-bit chunks. See the section on its integer outputs.

`MRG63k3a` is where that cost goes away, and the two rows read as one trade. Its `Float64` is 20% slower (5.10 ns against 4.25) — the step is a 128-bit multiply where MRG32k3a's is a `Float64` one — while its `UInt64` is 2.0× faster (10.76 ns against 21.06) and its `UInt32` 2.1× faster (5.04 against 10.48), because a word is two 32-bit chunks rather than four 16-bit ones. Per random *bit* it is ahead everywhere: 0.081 ns against 0.133 for the double, where 63 bits come out of the same one step.

In bulk it is the recurrence-based generator that loses least: 184 against 196 million `Float64`/s is a drop of 6%, where MRG32k3a drops 31% and the xoshiro families about a third. Its `UInt64` is flat at 93 either way. Why the drop is so much smaller here than for its sibling is not something these figures settle, so it is reported and not explained.

The counter-based generators are the ones that gain from filling an array: `rand!` produces whole blocks straight into it, so the counter, the block buffer and the index are touched once per block instead of once per draw. Philox4x64-10 goes from 203 to 359 million `Float64` per second, a factor of 1.77. The recurrence-based generators have no such block to exploit and are *slower* in bulk than in the accumulator loop, by the cost of the stores.

Two combinations get no block path and fall back to the scalar loop: `UInt32` from a 64-bit family, and any 64-bit draw taken while the generator sits mid-block because the caller has been mixing widths.

Every generator draws with zero allocations, in both paths, as do `short_jump!` and `long_jump!`.

GPU fills

Reproduce with `julia -O3 scripts/benchmarks/gpu_fills.jl`, which states its method in its header. Figures below are one run on an NVIDIA RTX A1000 (CUDA.jl 6.4.2), a card power-limited at 50 W whose rates move by 10 to 20% from one run to the next: fills of 2^{26} elements, billions of elements per second.

Generator	`rand!` `Float64`	`rand!` `Float32`	`randn!` (any type)	`randexp!`	`randn32!`
`Philox4x32-10`	19.3	41.3	0.60	1.45	14.9
`Philox4x64-10`	11.5	27.7	0.60	1.45	12.5
`Threefry4x32-20`	7.1	14.2	0.60	1.44	10.6
`Threefry4x64-20`	5.5	6.5	0.60	1.32	5.9

For scale, on the same card: writing the array with `fill!` runs at 23.5 (`Float64`) and 47.0 (`Float32`) billion elements a second, the memory bound; CUDA.jl's default `rand!` (cuRAND) at 20.7 and 45.1, and its `randn!` (Box-Muller) at 1.42 and 41.2; this package's `randn_boxmuller!` at 1.44 and 24.6.

What the table says. A uniform fill from Philox4x32 is at the memory bound, as cuRAND's is; the other three ciphers are bound by their own arithmetic — 64-bit multiplications and twenty rounds cost a GPU more than a CPU. Every `randn!` is at 0.6 whatever the generator, and whatever the output type: it is the `Float64` inversion, on a card that computes `Float64` at a small fraction of its `Float32` rate, and that limit is the card's — cuRAND's `Float64` Box-Muller meets it at 1.4. `randn32!` keeps inversion in `Float32` throughout and runs twenty to twenty-five times faster.

A large uniform or `randn32!` fill runs one thread per block of the generator, computing each block once and writing every draw whose words lie in it; one thread per element, as before, computed the block once per word, four times for a `Float32` fill. That halved the `Float32` rate of Philox4x32, and cost the 64-bit ciphers a factor of three to four in `Float32`. Below 2^{16} blocks, and for the `Float64` inversions at every size, one thread per element is kept: it finishes small fills sooner, and the inversion, not the cipher, is then the work. Either kernel gives the same values; the test suite runs both against the CPU.

A small fill is launch-bound: about 3 µs for a thousand uniforms from a process pinned to a performance core, 4 µs from an efficiency core — host time, which a GPU does not shorten.

Part VII

Validation

Chapter 7

Statistical Validation (TestU01)

`RandomDataStreams.jl` integrates seamlessly with TestU01, the industry-standard C library for empirical statistical testing of RNGs. It calls that library directly, through the `TestU01_jll` artifact and a layer of its own in `test/tu01.jl`; How the batteries are driven says what that layer is and why it is not `RNGTest.jl`.

The validation is deliberately split in two. The **package test suite** runs SmallCrush only, over three suites, and answers the question "does this installation work" in about six minutes — nine generators times three suites at some eleven seconds a battery, plus a minute for everything else. **Crush and BigCrush run on demand** through `scripts/testu01/validate.jl`, because they take hours to days; see Running Crush and BigCrush.

7.1 In the test suite: SmallCrush

A subset of the TestU01 batteries, **SmallCrush**, is automatically executed on all supported generators (`MRG32k3a`, `MRG63k3a`, `Xoshiro256+`, `PCG64`, `PCG64DXSM`, `Philox4x32-10`, `Philox4x64-10`, `Threefry4x64-20` and `Threefry4x32-20`) during the standard test suite.

It is the C battery itself, `bbattery_SmallCrush`, and the suite asserts on all 15 statistics it reports — read from `bbattery_pVal[]`, so they are the doubles the library computed. A p-value outside $[0.001, 0.999]$ fails the test and is printed with its name and its value. Over a whole campaign such a line is expected from chance and means nothing on its own (see Reading a summary report); on one CI run of nine generators it is rare enough to be worth a human's attention.

You can trigger this locally by running:

```
using Pkg
Pkg.test("RandomDataStreams")
```

7.2 Dependence between streams

A battery run on a single stream says nothing about whether the *streams* are independent of each other. Following L'Ecuyer et al. (2021, Sec. 6) — "it is also important to test the dependence between those streams [...] one can construct sequences that take a few values from each stream for a certain number of streams, in a round-robin fashion" — the test suite also runs SmallCrush on a sequence built by interleaving 64 streams, one value at a time, for each of the nine generators. For a counter-based generator this is what exercises the key schedule: a schedule handing out structurally related keys shows up here and not in a battery run on a single stream.

The heavier configurations recommended by the paper (up to 1024 streams and up to 8 values per stream) are reached through `--streams` and `--values` of `scripts/testu01/validate.jl`. A campaign runs the 64 × 1 shape unless `--shapes` names others, one job per shape:

```
./scripts/run-campaign.sh run --battery=bigcrush --suite=interleaved \
    --shapes=16x8,256x2,1024x1 --out=$HOME/bigcrush-shapes
```

Substreams. The paper describes two ways of making streams, and the interleaved suite tests one: every stream its own key, or its own jump, from `next_stream!`. The other is the substreams of a single stream, from `next_substream!` — for a counter-based generator one key and consecutive values of the counter's high half. That is the construction cuRAND uses: its Philox*4321*0 subsequences are `PhiloxRNG`'s substreams, word for word, and its MRG32k3a subsequences are MRG32k3a's substreams, `2^76` apart. The `substreams` suite interleaves substreams 1 to `--streams` of the first stream, in the same shapes, and replays a suspect on substreams of that same stream past those it used. A campaign runs it only when asked: `--suite=all` still means the three suites every earlier campaign ran, so their results directories resume unchanged.

```
./scripts/run-campaign.sh run --battery=bigcrush --suite=substreams \
    --shapes=16x8,64x1,256x2,1024x1 --generator=Philox4x32-10,MRG32k3a
```

Platform limitation

The batteries do not run on **Apple Silicon**. TestU01 is handed its callback through `@cfunction($f, ...)`, which needs an executable trampoline for the closure, and macOS on aarch64 forbids memory that is both writable and executable — Julia raises `cfunction: closures are not supported on this platform`. This affects every generator and every battery, and is a property of the platform rather than of any package here.

The test suite probes the capability and skips the three batteries with a message when it is missing, so the rest of the suite still runs; the on-demand harness refuses with the same explanation. Everything else in the suite, including the reference vectors, runs everywhere.

7.3 The integer paths, at the bit level

The Crush batteries examine the 30 most significant bits of the `U(0,1)` outputs, so a battery run on the float says nothing about how a generator builds a machine integer. Where the integer *is* the native output — Philox 4x32, Threefry 4x32 — that is the same stream. Where it is a construction, it needs testing for itself, and `test/test_bits.jl` runs SmallCrush on the bit stream of: `MRG32k3a` in `UInt32` and in `UInt64` (16-bit chunks of a non-power-of-two modulus), `MRG63k3a` in the same two types (32-bit chunks of the same construction, with a modulus just under `2^63`), the 64-bit counter-based families in `UInt32` (the low half of a cipher word), `PCG64` and `PCG64DXSM` in `UInt32` (the low bits of a permuted output – DXSM ends in a multiplication, whose low bits depend on fewer input bits than its high ones), and `Xoshiro256+` in `UInt32` (the low bits of an additive scrambler).

This battery earns its place: a `UInt64` construction for MRG32k3a that passed every `U(0,1)` battery failed here with six p-values at zero. Note also that passing does not clear the lowest bits of the + scramblers, whose linear weakness Blackman & Vigna document and SmallCrush does not resolve.

The batteries that were built for bits

SmallCrush over a `UInt32` stream is a smoke test on the integer path, not the instrument designed for it. TestU01's own bit-level batteries are **Alphabit** and **Rabbit**, and both run on demand:

```
julia scripts/testu01/validate.jl --battery=alphabit --suite=bits
julia scripts/testu01/validate.jl --battery=rabbit --suite=bits --bits=4294967296
```

They differ from the Crush batteries in taking a *size*: `--bits` is how much output to consume, 2^{30} by default. At that size Alphabit costs about half a minute — nine tests, seventeen statistics — and Rabbit about ten minutes, for twenty-six tests and some forty. Both are close to linear in the size above a fixed overhead, so four times the bits costs four times the time; Rabbit skips its larger `MatrixRank` tests when the size is too small to feed them, which is one reason not to shrink it far.

Alphabit examines bits *r+1* through *r+s* of each word, and the `bits` suite delivers whole `UInt32`s, so the default (0, 32) uses all of them. Nothing stops either battery running on the `single` and `interleaved` suites — `unif01` extracts bits from a double as readily as from a word — and the campaign matrix covers all three.

A replay has to consume the same number of bits as the run it replicates, so `validate.jl` records `# bits = N` in the log header and `replay.jl` reads it back from there; `--bits` overrides it.

7.4 Running Crush and BigCrush

These are **not** part of `Pkg.test()` and must not become part of it. The package test suite answers "does this installation work", and SmallCrush at about a minute per battery is the most that belongs there. Crush takes hours and BigCrush the better part of a day per generator: that is a validation exercise, for a release or for a paper, not an installation check.

`scripts/testu01/validate.jl` runs any battery over any of the three suites, on demand:

```
# what would run, without running it
julia scripts/testu01/validate.jl --list --battery=bigcrush --suite=all

# one generator, one battery, one suite
julia scripts/testu01/validate.jl --battery=crush --generator=Xoshiro256p --suite=bits

# the full single-stream sweep
julia scripts/testu01/validate.jl --battery=bigcrush --suite=single
```

option	values	default
`--battery`	`smallcrush`, `crush`, `bigcrush`, `alphabit`, `rabbit`	`smallcrush`
`--suite`	`single`, `interleaved`, `bits`, `all`	`single`
`--generator`	a generator name, or `all`	`all`
`--streams`, `--values`	shape of the interleaved suite	64, 1
`--bits`	output consumed by `alphabit` and `rabbit`	2^{30}
`--out`	directory for logs	`testu01-results`
`--list`	print the plan and exit	
`--quiet`	summary only, no per-test reports	

Each run writes three things. TestU01's own report goes to its own log: per-test reports are on by default, because they are the canonical artefact to archive and because they make a long run observable — the log fills as tests complete, so progress can be followed with `tail -f` and an interrupted run still leaves its completed tests behind. `summary.tsv` gets one line for the run: battery, suite, generator, wall time, Julia version, CPU, the commit, and how many statistics the battery reported, singled out and failed outright, so the question "did anything happen" is answered without opening a log. `pvalues.tsv` gets one line per statistic — index, name, p-value, class — taken from `bbattery_pVal[]` rather than from the printed report, which shows p-values in TestU01's own formatting (`eps`, `1 - 3.0e-9`). Classifying against 10^{-10} is not something to do on text.

Run it under `nohup` or `tmux`:

```
nohup julia scripts/testu01/validate.jl --battery=bigcrush --suite=all > run.out 2>&1 &
tail -f testu01-results/bigcrush-*.log
```

7.5 Reading a summary report

Every battery ends with a list of the p-values it singled out. That list is not a verdict, and reading it as one is the easiest way to publish a wrong claim.

The threshold TestU01 prints against is `gofw_Suspectp`, whose default is 0.001: "any p-value less than α or larger than 1−α is considered suspect and is *singled out*". It is a **printing** threshold. A battery of Crush's size reports 144 statistics, so a campaign of 51 runs produces 7344 of them, and $0.002 \times 7344 \approx 15$ lines are expected from chance alone — before any generator has done anything wrong. (The statistics within a run are not independent, as several tests compute more than one from the same stream; that widens the spread but leaves the expected count unchanged.)

The decision rule is in the TestU01 guide, under *Rejecting H_0*:

> When a p-value is extremely close to 0 or to 1 (for example, if it is less than 10^{-10}), one can obviously conclude that the generator fails the test. If the p-value is suspicious but failure is not clear enough, (p = 0.0005, for example), then the test can be replicated independently until either failure becomes obvious or suspicion disappears.

So there are three outcomes, not two:

	criterion	what to do
failure	$p < 10^{-10}$ or $p > 1 - 10^{-10}$	report it; no replay needed
suspect	outside [0.001, 0.999] but inside $[10^{-10}, 1 - 10^{-10}]$	replay before calling it anything
pass	anything else	TestU01 does not print it

Fix the number of replications before looking at them, and report the p-values rather than a reject/do-not-reject verdict — the guide is explicit that this "provides more information".

Replaying the suspects

`scripts/testu01/replay.jl` re-applies only the tests a run singled out:

```
# one run: the newest complete log in that directory, its flagged tests, 3 times each
julia scripts/testu01/replay.jl --log=<out>/runs/crush-interleaved-Threefry4x64_20

# or by hand
julia scripts/testu01/replay.jl --battery=crush --suite=bits \
      --generator=Xoshiro256ss --tests=19 --reps=5

# the whole campaign at once
./scripts/run-campaign.sh replay --battery=crush --reps=3
./scripts/run-campaign.sh replay --battery=crush --dry-run   # what it would do
```

Three things about it are worth knowing.

It costs minutes, not hours. `bbattery_RepeatCrush(gen, rep)` applies test *i* of the battery `rep[i]` times and skips the rest. Resolving the three suspects of one Crush run took 106 seconds against the 30 minutes the run itself had taken — which is what makes the same discipline affordable at BigCrush, where a re-run would otherwise cost hours.

The output must be disjoint. `seed_words` is a fixed seed on purpose, so a run can be reproduced exactly; re-running a suite the ordinary way therefore reproduces the same p-values and settles nothing. `--skip`

advances the stream generator past everything the original consumed — 1024 streams by default, well clear of the 64 the interleaved suite takes by default, and as many streams as a larger shape takes — so the replay is the independent replication the guide asks for. An interleaved suspect is replayed in the shape it arose in, which the replay reads from the run's log. This is the package's own stream facility doing the work the method requires.

The p-values come back exact. A replay writes `replay-pvalues.tsv` beside its log, with every statistic it produced — not only the ones a battery would single out, since what settles a suspect is the spread of the replications, and the guide is explicit that reporting the p-values "provides more information" than a verdict. The console prints the same list.

The number beside a suspect is the test, not the statistic. A battery reports more statistics than it has tests (144 against 96 for Crush), and TestU01 prints the *test* number, so several lines can carry the same one: five lines reading `10 RandomWalk1 ...` are one test with five statistics, not five tests. That number goes straight into `rep`, which is why no mapping table is needed — but it also means "three suspect lines" and "three suspect tests" are different counts.

The Crush campaign, worked through

The 2026 Crush campaign — seventeen generators, three suites, 51 runs, with Crush itself on the `bits` suite, as the campaign was shaped before Alphabit and Rabbit were reachable — gives the rule something to bite on. (Its `U(0,1)` results for the fifteen word-based generators predate the switch to the open interval; they were measured again, below.) **No p-value came within four orders of magnitude of 10^{-10}**, so nothing was a failure outright. Eighteen lines were singled out across fifteen runs, against ≈15 expected: an unremarkable count, but a count is not an answer for any particular line.

So all eighteen were replayed, three times each on disjoint streams — 54 replications, about twelve minutes of work. **Not one produced a p-value outside [0.001, 0.999].** The suspects were chance, every one of them, which is what a campaign of this size is expected to manufacture and exactly what the replay exists to establish rather than assume.

Two of them had looked worth the trouble. The most extreme p-value, 6.7×10^{-6} (`Xoshiro256ss`, bits, ClosePairs mNP), is more extreme than one would typically see among 7344 statistics — probability ≈ 9%. And `Threefry4x64-20` carried five of the eighteen across two of its three suites, three of them in the interleaved run alone, where three or more in one run arises in about 15% of campaigns this size. Replayed on streams 1025 and up, those three gave:

test	original	replay 1	replay 2	replay 3
19 ClosePairs mNP2, t = 3	$1 - 2.9 \times 10^{-5}$	0.9988	0.11	0.32
60 MatrixRank, 1200 × 1200	1.3×10^{-4}	0.25	0.96	0.73
90 HammingIndep, L = 1200	8.6×10^{-4}	0.79	0.65	0.45

Suspicion gone, in 106 seconds against the 30 minutes the run itself took. That is the shape the write-up should take for each suspect: the original p-value, the replications, and the p-values themselves — not a verdict.

After the switch to the open interval

The `U(0,1)` suites of the fifteen word-based generators — xoshiro and xoroshiro, PCG, the counter-based families — measured the conversion they used before every float draw moved to `(0, 1)` (see Implementation Notes). Those thirty runs, `single` and `interleaved`, were repeated on the new conversion: Crush, at commit `3f67bc0`, on an Intel Core i5-13500H, five at a time, ten hours in all. The MRG families, whose `Float64` draws did not change, were not re-run. This was a first pass with Crush; the BigCrush sweep followed.

Of 4320 statistics, **none came within five orders of magnitude of 10^{-10}** — the most extreme lay 4.3×10^{-5} from 1 — and ten lines were singled out, against ≈8.6 expected from chance. All ten were replayed three times on disjoint streams; not one replication left [0.001, 0.999]:

generator	suite	test	original	replay 1	replay 2	replay 3
PCG64	single	CollisionOver, t = 20	$1 - 4.3 \times 10^{-5}$	0.20	0.73	0.61
PCG64	single	RandomWalk1 J, L = 10000	2.7×10^{-4}	0.97	0.92	0.52
Xoshiro256pp	single	WeightDistrib, r = 24	1.4×10^{-4}	0.12	0.48	0.54
Xoshiro256pp	interleaved	SerialOver, t = 4	3.6×10^{-4}	0.62	0.75	0.76
Xoroshiro128ss	single	MatrixRank, 300 × 300	$1 - 1.6 \times 10^{-4}$	0.81	0.63	0.32
Xoroshiro128ss	single	RandomWalk1 J, L = 10000	$1 - 1.7 \times 10^{-4}$	0.041	0.33	0.088
Xoshiro256p	interleaved	ClosePairs mNP1, t = 7	$1 - 7.1 \times 10^{-4}$	0.86	0.20	0.63
Xoshiro256ss	single	ClosePairs NJumps, t = 2	$1 - 7.4 \times 10^{-4}$	0.25	0.69	0.81
Threefry4x64-20	single	AutoCor, d = 30	$1 - 9.1 \times 10^{-4}$	0.043	0.65	0.21
Xoroshiro128p	single	RandomWalk1 J, L = 90	8.4×10^{-4}	0.17	0.70	0.46

The same picture as the first campaign: a count at the level chance predicts, and suspects that scatter on independent output.

BigCrush over the whole matrix

The full matrix — the seventeen generators, BigCrush on the `single` and `interleaved` suites, Alphabit and Rabbit at 2^{30} bits on `bits`, 68 runs — was run at commit 2b485cd (v0.3.0), from a clean tree, on an Intel Core Ultra 9 285 (8 performance and 16 efficiency cores, no SMT) with Julia 1.12.1, 24 runs at a time. A BigCrush run took 2.6 to 5.0 hours, mostly according to the kind of core it landed on, with a peak resident set of 1.3 GB; Alphabit took half a minute and Rabbit two to five. The sweep took eight hours of wall-clock time, after a calibration run of 2.6 hours.

battery, suite	runs	statistics	outside [0.001, 0.999]	failures
BigCrush, `single`	17	2720	3	0
BigCrush, `interleaved` (64 streams × 1)	17	2720	5	0
Alphabit, `bits`	17	289	1	0
Rabbit, `bits`	17	680	2	0

Of 6409 statistics, **none was a failure outright**: the most extreme lay 2.8×10^{-6} from 1 (`PCG64DXSM`, Rabbit, MultinomialBitsOver), four orders of magnitude short of 10^{-10}. Eleven lines were singled out, against ≈12.8 expected from chance. All eleven were replayed three times on streams 1025 and up, and **none of the 33 replications left [0.001, 0.999]**:

A replay recomputes every statistic of the tests it re-applies, 60 here, and one that had not been singled out left the interval once: ClosePairs NP, t = 5, for `Xoroshiro128p` interleaved, 0.21 in the original run and 0.58,

generator	suite	test	original	replay 1	replay 2	replay 3
PCG64DXSM	bits, Rabbit	MultinomialBitsOver	$1 - 2.8 \times 10^{-6}$	0.0033	0.96	0.79
Xoshiro256p	bits, Rabbit	MultinomialBitsOver	1.5×10^{-4}	0.31	0.87	0.94
Xoshiro512pp	single	CollisionOver, t = 21	1.5×10^{-4}	0.97	0.37	0.12
Philox4x32-10	single	WeightDistrib, r = 0	$1 - 1.7 \times 10^{-4}$	0.035	0.41	0.30
Xoshiro256p	single	Gap, r = 0	2.4×10^{-4}	0.32	0.84	0.12
Threefry4x32-20	bits, Alphabit	RandomWalk1 J, L = 64	4.6×10^{-4}	0.76	0.67	0.23
MRG63k3a	interleaved	SampleMean, r = 10	$1 - 6.4 \times 10^{-4}$	0.12	0.13	0.57
Xoroshiro128p	interleaved	ClosePairs NJumps, t = 5	7.3×10^{-4}	0.74	0.62	0.36
Xoroshiro128p	interleaved	CollisionOver, t = 14	$1 - 4.2 \times 10^{-4}$	0.29	0.51	0.71
Philox4x32-10	interleaved	HammingCorr, L = 1200	$1 - 8.0 \times 10^{-4}$	0.013	0.58	0.35
Xoshiro256ss	interleaved	CollisionOver, t = 7	$1 - 6.2 \times 10^{-4}$	0.63	0.74	0.12

0.28 and 0.9994 in the replays — one extreme value in four, for a statistic unremarkable the first time, against ≈0.12 such lines expected among 60.

The interleaved suite in four shapes

L'Ecuyer et al. (2021, §6) suggest a power of two between 16 and 1024 streams, and one to eight values from each in turn. The 28 power-of-two shapes in that range would be 476 BigCrush runs; three were fixed instead, before anything was computed — (16, 8), (256, 2) and (1024, 1), a diagonal from a few streams each giving a run of consecutive values to the most stream starts side by side, one value each. They ran at commit a869931, for the seventeen generators: 51 BigCrush runs of the `interleaved` suite, on the same machine and 24 at a time, from 2.6 to 5.1 hours each. A suspect is replayed in the shape it arose in.

shape (streams × values)	runs	statistics	outside [0.001, 0.999]	failures
16 × 8	17	2720	9	0
64 × 1 (the sweep above)	17	2720	5	0
256 × 2	17	2720	4	0
1024 × 1	17	2720	5	0

Of the 8160 statistics of the three new shapes, **none was a failure**: the most extreme was 2.8×10^{-5} (`Xoshiro256ss`, 16 × 8, CollisionOver, t = 21). Eighteen were singled out, against ≈16.3 expected, and each was replayed three times in its own shape, on streams 1025 and up:

The replays recompute 135 statistics, and one left the interval, against ≈0.27 expected: the second replay of Permutation, t = 7, for `Xoshiro256p` at 1024 × 1, on the same side as the original. Together with SampleCorr, k = 2, for `Xoroshiro128pp` — the only (generator, statistic) pair singled out in two shapes, against ≈0.03 such pairs expected, and whose last replay at 16 × 8 came close on the other side — that left three series that had not scattered cleanly enough to stop at three. Each was replicated twenty more times, on streams 2049 and up (`replay.jl --skip=2048`), disjoint from the run and from the first replay:

generator	shape	test	original	replay 1	replay 2	replay 3
Xoshiro256ss	16 × 8	CollisionOver, t = 21	2.8×10^{-5}	0.40	0.67	0.32
Xoroshiro128ss	16 × 8	MatrixRank, L = 5000	6.9×10^{-5}	0.41	0.86	0.59
Philox4x32-10	16 × 8	CollisionOver, t = 14	8.3×10^{-5}	0.044	0.58	0.38
Xoroshiro128ss	16 × 8	RandomWalk1 J, L = 10000, r = 0	1.4×10^{-4}	0.68	0.62	0.53
Xoshiro512pp	256 × 2	ClosePairs mNP2, t = 3	$1 - 2.7 \times 10^{-4}$	0.71	0.39	0.73
MRG32k3a	16 × 8	CollisionOver, t = 21	$1 - 4.6 \times 10^{-4}$	0.16	0.72	0.20
MRG63k3a	256 × 2	MaxOft, t = 8	$1 - 5.1 \times 10^{-4}$	0.49	0.44	0.21
Xoshiro256ss	1024 × 1	WeightDistrib, r = 28	5.9×10^{-4}	0.076	0.68	0.21
Xoshiro512pp	256 × 2	RandomWalk1 J, L = 50, r = 25	6.2×10^{-4}	0.75	0.37	0.26
Xoroshiro128pp	1024 × 1	SampleCorr, k = 2	6.3×10^{-4}	0.34	0.16	0.13
Xoshiro512p	1024 × 1	BirthdaySpacings, t = 7	6.7×10^{-4}	0.45	0.57	0.63
Xoroshiro128pp	16 × 8	SampleCorr, k = 2	7.6×10^{-4}	0.76	0.64	$1 - 1.1 \times 10^{-3}$
MRG32k3a	16 × 8	RandomWalk1 J, L = 10000, r = 15	$1 - 7.6 \times 10^{-4}$	0.029	0.75	0.66
PCG64	16 × 8	ClosePairs mNP2S, t = 16	8.3×10^{-4}	0.27	0.31	0.83
Xoroshiro128p	256 × 2	MatrixRank, L = 5000	8.8×10^{-4}	0.064	0.023	0.20
PCG64DXSM	16 × 8	MatrixRank, L = 30, r = 0	$1 - 9.0 \times 10^{-4}$	0.76	0.67	0.34
Xoshiro256p	1024 × 1	Permutation, t = 7	9.0×10^{-4}	0.17	**8.0×10^{-4}**	0.13
PCG64	1024 × 1	RandomWalk1 J, L = 10000, r = 0	9.6×10^{-4}	0.77	0.21	0.20

generator	shape	test	min	median	max	Kolmogorov–Smirnov
Xoshiro256p	1024 × 1	Permutation, t = 7	0.034	0.47	0.997	D = 0.14, p ≈ 0.81
Xoroshiro128pp	16 × 8	SampleCorr, k = 2	0.14	0.51	0.92	D = 0.14, p ≈ 0.78
Xoroshiro128pp	1024 × 1	SampleCorr, k = 2	0.0020	0.54	0.95	D = 0.11, p ≈ 0.97

All sixty stayed inside [0.001, 0.999], and each series is consistent with uniformity: **no suspect persists**. With the 64 × 1 sweep, the interleaved suite has passed BigCrush in four shapes, 10 880 statistics, for every generator.

7.6 Long campaigns: running for days without a session

`validate.jl` runs one battery in one process, serially. That is the wrong shape for a BigCrush sweep: the full matrix is 68 runs — seventeen generators on the `single` and `interleaved` suites, plus Alphabit and Rabbit on

the `bits` suite — most of them taking the better part of a day, so a single process means weeks of sequential work in which one crash loses everything and a lost SSH session ends the campaign.

`scripts/testu01/campaign.jl` runs the same matrix as independent OS processes:

```
# what would run, and what is already done
julia scripts/testu01/campaign.jl --battery=bigcrush --status

# measure one job before committing to the sweep
julia scripts/testu01/campaign.jl --battery=bigcrush --calibrate

# the sweep, six jobs at a time
julia scripts/testu01/campaign.jl --battery=bigcrush --jobs=6
```

`--battery` is required: there is deliberately no default, so that a stray invocation cannot start a two-week sweep.

A campaign need not run the whole matrix. `--suite` keeps some of the three suites, and `--shapes=<streams>x<values>[,.` gives the interleaved suite one job per shape. `run-campaign.sh` records both next to the results, so that `status`, `stop` and the watchdog work against the matrix that was started, and it refuses a different matrix in the same directory rather than mix the two. With `--jobs` given, `--skip-calibration` starts the sweep without first running one job alone to measure it — at BigCrush, hours during which every other core waits.

The `bits` leg does not run `--battery`. Crush and BigCrush examine the 30 most significant bits of a `U(0,1)` output, so over an integer stream they are the wrong instrument — running one there was what the campaign did for want of an alternative. A campaign now runs **Alphabit and Rabbit** on the `bits` suite whatever `--battery` says, which is where the extra seventeen runs come from. `--bits-battery=<name>[,<name>]` overrides that, and passing the same name as `--battery` restores the old uniform matrix; `--bits=<n>` sets the size those two consume. `validate.jl` stays orthogonal — it runs any battery over any suite, and this pairing is the campaign's policy, not the library's.

Resuming is the point. `summary.tsv` records one line per finished `(battery, suite, generator)`, and for the interleaved suite per shape. Re-running the same command skips those and does the rest, so an interruption costs only the jobs actually in flight — never the weeks already spent. Each job also writes its own `summary.tsv` in its own directory, which the driver reconciles, so even a driver killed mid-flight loses nothing that finished.

Stopping cleanly. `touch <out>/STOP` stops new jobs from starting and waits for the running ones, which is how to end a campaign without discarding a battery that is twenty hours in. Ctrl-C does the same.

Surviving logout and reboot. `setsid nohup` survives a logout; `tmux` does too and lets you reattach, unless it moves its panes into the systemd user manager (below); neither survives a reboot or restarts a driver that died. For a campaign measured in weeks, use the systemd *user* unit in `scripts/testu01/randomdatastreams-campaign.service`:

```
loginctl enable-linger $USER          # user services keep running after logout
cp scripts/testu01/randomdatastreams-campaign.service ~/.config/systemd/user/
$EDITOR ~/.config/systemd/user/randomdatastreams-campaign.service   # set paths
systemctl --user daemon-reload
systemctl --user start randomdatastreams-campaign
journalctl --user -u randomdatastreams-campaign -f
```

The unit stops with `SIGINT` and a six-hour stop timeout, so `systemctl --user stop` lets the driver wait for its children rather than killing them, and `Restart=on-failure` brings back a driver that died — it resumes from `summary.tsv` like any other invocation.

When `enable-linger` is not yours to run

That first line is the one that needs an administrator. Without it, systemd kills user services at logout, which is exactly what a two-week sweep must survive. On a shared machine the answer is often "no", so `scripts/run-campaign.sh` can supervise the campaign without systemd:

```
./scripts/run-campaign.sh run --battery=crush --supervisor=detached --watchdog
./scripts/run-campaign.sh status --battery=crush      # no flag needed afterwards
./scripts/run-campaign.sh stop   --battery=crush
```

`--supervisor=detached` starts the driver with `setsid nohup`, in a session of its own, and records its pid in `<out>/driver.pid`; its output goes to `<out>/supervisor.log`. `stop` sends `SIGINT` to its process group, which holds the jobs as well — what Ctrl-C does in a terminal. `--supervisor=auto`, the default, uses systemd when linger is on or can be turned on, and falls back to `detached`. The choice is recorded in the output directory, so `status`, `stop` and `collect` find the campaign without being told again, and a `start` that names another supervisor records that one instead.

What neither gives you is what `--watchdog` restores, through a *user* crontab entry that needs no privileges:

	systemd unit	detached	detached + `--watchdog`
survives logout	with linger	yes, unless `KillUserProcesses=yes`	same
survives a reboot	yes	no	yes (`@reboot`)
restarts a driver that died	`Restart=on-failure`	no	yes, within 15 min
watch it work	`journalctl -f`	`tail -f <out>/supervisor.log`	same

tmux, where it still works. `--supervisor=tmux` runs the driver in a detached tmux session you can attach to, and was the fallback until it took a BigCrush sweep down. A tmux built with systemd support — 3.6 on the machine where it happened — moves each pane it spawns into a scope of its own under the systemd user manager, `user@<uid>.service`, so that the pane outlives the terminal it came from. Without linger that manager is stopped when your last session ends, and every scope under it with it; `KillUserProcesses=no` does not help, since it spares what stays in a session's scope and the pane has left it. The last logout of 2026-10-16 took the driver and the 24 BigCrush jobs it was running, some five hours in. `start` now asks the pane where it landed, from `/proc/<pid>/cgroup`, and when that is under the user manager with linger off it stops the session again, before a job has started, and says to use `detached`. A detached driver stays in the scope of the login or the cron job that started it.

Two details the script handles, both of which are silent failures otherwise. `KillUserProcesses=yes` in `logind.conf` kills a detached driver or tmux session at logout just as it kills an unlingered service, so neither is a workaround on such a machine — `run-campaign.sh` reads the setting and says so before you commit two weeks to it. And tmux keeps its socket under `/run/user/$UID`, a directory that is removed when your last session ends; the script points `TMUX_TMPDIR` at a directory under `$HOME` instead, so the session started from your login shell and the one the watchdog looks for are the same session.

The watchdog respects `touch <out>/STOP`: a campaign you stopped on purpose stays stopped. `stop` removes the crontab entry, and any crontab it edits is backed up to `<out>/crontab.backup` first — your other cron jobs are never touched.

A home directory that answers to a ticket

Where $HOME is an NFS mount secured with Kerberos — sec=krb5 in the output of findmnt -T ~ -o OPTIONS — the files answer to a *ticket*, not to your uid. When the ticket lapses, every process you own loses the directory at the same instant: the driver in the middle of writing a result, and cron before it can so much as read the watchdog's copy of this script. Tickets are measured in hours and campaigns in days, so this is not a corner case. It is what happens by default to a sweep left to run.

Nothing in the campaign reports it, because nothing in the campaign can still write anywhere to report it. From outside, it looks like a watchdog that quietly stopped firing; the system's cron log tells the real story, one line per attempt:

```
CROND[91468]: (CRON) ERROR chdir failed (/u/you): Permission denied
```

prepare recognises the case. It refuses to begin without a valid ticket, and gives the campaign **a ticket cache of its own**: a copy of yours, at /tmp/krb5cc_<uid>_randomdatastreams-<battery>, renewed every 30 minutes by a krenew that no login owns, and named in the launcher:

```
export KRB5CCNAME="FILE:/tmp/krb5cc_<uid>_randomdatastreams-bigcrush"
```

Your own cache will not do wherever each login gets one — /tmp/krb5cc_<uid>_XXXXXX, made by sshd or pam_sss and destroyed at logout. A driver started under a krenew that inherited it fails with No credentials cache found once you have disconnected; a krenew -b started from your shell goes the same way; and cron never sees a KRB5CCNAME at all, and looks for /tmp/krb5cc_<uid>, which nobody made. That is what held up the first BigCrush sweep for six hours one night, the calibration finished and every core idle, with the watchdog failing every fifteen minutes. The NFS client needs no telling: rpc.gssd looks in /tmp for a cache you own whose name starts with krb5cc_. The watchdog brings the campaign's krenew back if it dies while the ticket still holds, and stop destroys the cache, which existed only for the campaign. Where your cache is not a file (KEYRING:, KCM:) there is nothing to copy, and the driver runs under krenew on the cache you have; such caches usually outlive a login, but check that yours does before leaving a sweep to run.

Two limits are worth knowing before committing days to a sweep. First, **renewal has its own deadline**: a renewable ticket may be renewed only until the renew until that klist -f prints, typically some weeks after the kinit that created it, and a ticket that is not renewable at all cannot be rescued — prepare warns in both cases. prepare and start copy the ticket you have at the time, so a fresh login followed by either restarts that clock. Second, **a reboot clears /tmp** on most machines, and the campaign's cache with it: the watchdog's @reboot line then cannot so much as enter $HOME, and the campaign waits for you to log in and run start.

A lock that never wakes. One more thing goes wrong on such a home, and has nothing to do with tickets. Every job used to begin with Pkg.activate, which writes ~/.julia/logs/manifest_usage.toml under a pid lock and, while another process holds it, waits on a file watcher. Of 24 BigCrush jobs started within the same minute on an NFS home, six waited there for nine and a half hours, until they were killed: the watcher never fired. Julia prints a process's stacks on kill -USR1 <pid>, and again when SIGTERM kills it; all six stood at the same place. The driver resolves the environment once, before any job starts, so a job whose Manifest already points at the checkout now sets its active project directly and calls no Pkg at all. Only a stale Manifest still goes through Pkg, lock included, and the driver's own pass is what keeps it from being stale.

The alternative dissolves the question rather than answering it — put the checkout and the results on a filesystem that needs no ticket, such as local scratch space:

```
./scripts/run-campaign.sh prepare --battery=bigcrush --out=/var/tmp/$USER-bigcrush
```

Every result names the code that produced it. `summary.tsv` records the package version, the commit of `HEAD` and whether the working tree was clean, alongside the Julia version, the CPU and — for the interleaved suite — the number of streams and values per stream. Each battery log repeats it in its header. A campaign started from a dirty tree prints a warning before it begins, because a run measured in weeks cannot be repeated to find out afterwards which code produced it. The same line appears at the top of the throughput benchmark's report.

Do not benchmark and validate at the same time. The throughput figures in the implementation notes are minima over samples on an unloaded machine; a host running six BigCrush processes will produce numbers that measure the scheduler. Run `scripts/benchmarks/throughput.jl` first, pinned, then start the campaign. On a hybrid CPU (Intel 12th generation and later) pin the benchmark to a performance core with `taskset`; the batteries can use any core, since their results do not depend on how fast they were produced.

What BigCrush settled, and what it did not

Two things the suite cannot settle at SmallCrush level, and where the BigCrush sweep leaves them:

- **The low bits of the + scramblers.** Blackman & Vigna document the lowest bits of `xoshiro256+` and `xoroshiro128+` as linearly weak, and the `bits` suite hands exactly those bits to the batteries: a `UInt32` is the low half of the word. Alphabit and Rabbit at 2^{30} bits passed on all three + generators, and BigCrush passed on their `U(0,1)` output. That narrows the question without closing it: a pass is not a proof, and a weakness confined to the lowest bit or two can be diluted in a test of the whole 32-bit word.
- **Dependence between streams.** The interleaved suite is the construction L'Ecuyer et al. (2021) call for, and the paper says batteries for counter-based generators still need development. BigCrush passed on it for all seventeen generators in four shapes, from 16 streams of eight values to 1024 streams of one. For the counter-based generators each of those streams has a key of its own; the other construction the paper describes, the substreams of a single key — which is how cuRAND hands out its Philox streams — has not been run.

Results belong in the JOSS submission, not in CI.

7.7 How the batteries are driven

Every battery on this page is a direct call into the C library, through the `TestU01_jll` artifact and one layer of our own in `test/tu01.jl`. That layer is what the test suite and both scripts use: it builds and frees the `unif01_Gen`, runs the batteries, and reads their answers out of `bbattery_pVal[]`, `bbattery_NTests` and `bbattery_TestNames[]`. It lives under `test/` rather than beside the scripts because `Pkg.test()` has to be self-contained, while a script may reach anywhere in the checkout it belongs to.

It is deliberately small — batteries, the repeat batteries, the two printing globals, and the generator's lifetime — and everything in it is there because the alternative, RNGTest.jl, gave one of the following instead. The package depended on it until 2026-09; the record is kept because each point is a reason the code looks as it does, and because someone reaching for the wrapper again should know what they are taking on.

The batteries returned nothing. `smallcrushTestU01` and its siblings are `Cvoid` calls, and the three globals holding the answer are not exposed, so every p-value had to be recovered from the formatted report — as text, in TestU01's printing forms. That is a poor thing to classify against a threshold of 10^{-10}, and it made the log the only artefact a run produced. `pvalues.tsv` exists because the globals do.

The test suite could not run the real battery, for that same reason. It drove `RNGTest.smallcrushJulia`, a Julia reimplementation that applies SmallCrush's ten tests one at a time and returns their p-values — the only way to assert on them. It is not the same object: it keeps a subset of the statistics (`sstring_HammingIndep`

contributes its Mean alone), and it distributes the ten tests with pmap, so under `julia -p N` the generator closure would be serialised to each worker and every test would start from the same state — ten tests on one stream. The suite now runs bbattery_SmallCrush and asserts on all 15 statistics. That costs about 15% more per battery — 12.1 s against 10.6 s, measured on one generator in one process — which is the price of running the battery the literature names rather than something close to it.

The callback was not rooted. `@cfunction($f, ...)` returns a Base.CFunction, which Base documents as the "garbage-collection handle" that must outlive every C call through the pointer; `RNGTest.Unif01` builds it in a local and drops it when the constructor returns. The callback allocates, so the GC does run during a battery. It was never seen to crash — but nothing made it safe, and a battery that dies at hour eighteen of a BigCrush costs a day. Gen holds the handle and the closure for as long as the generator lives.

A `Function` generator lost its first 100 values. `Unif01(f::Function, name)` drew a hundred to check the return type and the range before TestU01 saw the generator, and each test of `smallcrushJulia` paid it again, since every test built its own. The layer checks the *inferred* return type instead, which costs nothing, catches the same mistake — a callback inferred as Any is a segfault, not a type error, once C is calling it — and leaves the stream at its origin.

`swrite_Basic` was cleared at load, and written back eight bytes at a time. The flag is a `lebool`, which `gdef.h` typedefs to `int`; RNGTest stores a pointer-sized value into it, as did the copy of that trick this repository was carrying. `TU01.verbose!` writes a `Cint`.

The repeat and bit-level batteries had no wrapper at all. `bbattery_Repeat*` is what makes Replaying the suspects affordable, and `Rabbit`, `Alphabit` and `BlockAlphabit` are the batteries actually meant for a bit source, and the `bits` suite had been running SmallCrush over a `UInt32` stream for want of them. `--battery=alphabit` and `--battery=rabbit` now run them, replays included; see The batteries that were built for bits. `BlockAlphabit` is in the layer but not wired to the command line: its repeat form takes a block width as well, and nothing here has asked for it yet.

Two rules the layer enforces that the C library only documents. **One generator at a time**: TestU01 keeps its state in globals and crashes if a second `unif01_Gen` is created, so Gen refuses to build one and names the generator still live; it is also why `campaign.jl` runs its matrix as independent OS processes rather than as tasks. **The callback must be inferred**, as `Float64` for a `U(0,1)` generator or `UInt32` for a bit generator — the generator tables here have an abstract element type, so every callback goes through a function barrier, and Gen rejects one that did not.

What the layer does not cover is the individual tests: the `s*` modules hold some fifty of them, and nothing here calls one directly, because a battery is what this package validates against. Should one ever be needed, the same artifact ships `libtestu01extractors`, the shim that reads a p-value out of an opaque `sres_*` struct, which is the larger part of what RNGTest is for.

Part VIII

Generator Comparison

Chapter 8

Choosing a generator

RandomDataStreams.jl ships two kinds of generator. They differ in how a stream is defined, which is what makes them suited to different jobs — throughput is the smaller part of the choice.

Recurrence-based generators (MRG32k3a and MRG63k3a, the xoshiro/xoroshiro families) hold a state and a transition function. There is one orbit, and a stream is a *segment* of it: `next_stream!` jumps a fixed, huge distance along the orbit, and non-overlap is a statement about those distances.

Counter-based generators (Philox, Threefry) hold a counter and a key, and produce a block as a keyed bijection of the counter, `R = b_K(C)`. A stream is a *key*: different streams use different bijections, so their outputs cannot overlap by construction rather than by distance. Position within a stream is an index, not a state, so any draw can be recomputed from `(key, substream, i)` alone — see Streams & Substreams.

	Recurrence-based	Counter-based
Stream =	a segment of one orbit	a distinct key
Non-overlap because	the jump distance exceeds any usable run	distinct keys are distinct bijections
Arbitrary jump	polynomial or matrix computation	one counter addition
Draw at index `i`	requires jumping there	computable directly
Speed	~5× faster per draw	block of `N` words amortises the bijection

8.1 Recurrence-based generators

(* means any of the +, **, ++ scramblers. `PCG64DXSM` has the same structure as `PCG64`, with a different output permutation and multiplier.)

8.2 Counter-based generators

Throughput figures come from the Implementation Notes, which state the measurement method and the machine; read them as ratios.

8.3 When to choose MRG32k3a

- You need **provably non-overlapping streams with published, peer-reviewed parameters** (L'Ecuyer et al. 2002), e.g. to stay compatible with simulation libraries built on that model (SSJ, Simio, Arena-style stream ids...).

Property	MRG32k3a	MRG63k3a	PCG64	Xoroshiro128*	Xoshiro256*	Xoshiro512*
State (bits)	192 (6 × Int64)	384 (6 × Int64)	128	128	256	512
Period	≈ 2^191	≈ 2^377	2^128	2^128 − 1	2^256 − 1	2^512 − 1
Native output	`Float64` (0,1), ~32-bit resolution	`Float64` (0,1), ~63-bit resolution	`UInt64`	`UInt64`	`UInt64`	`UInt64`
`Float64` throughput	236 M/s	196 M/s	504 M/s (DXSM 561)	≈ 599–697 M/s	≈ 688–788 M/s	≈ 523–570 M/s
`UInt64` throughput	47 M/s (4 steps)	93 M/s (2 steps)	599 M/s (DXSM 743)	≈ 818–941 M/s	≈ 1032–1242 M/s	≈ 725–798 M/s
Stream mechanism	matrix power A^2^127 on the seed	matrix power A^2^250 on the seed	closed-form LCG jump (2^32(2^64+1)+1)	`long_jump!` polynomial (2^96)	`long_jump!` (2^192)	`long_jump!` (2^384)
Substream mechanism	matrix power A^2^76	matrix power A^2^150	closed-form (2^64+1)	`short_jump!` (2^64)	`short_jump!` (2^128)	`short_jump!` (2^256)
Backward jumps	yes (`advance_state!`)	yes (`advance_state!`)	yes, same cost as forward	yes (`advance_state!`, GF(2) polynomials)	yes (`advance_state!`)	yes (`advance_state!`)
Arbitrary-jump cost	O(e) matrix products	O(e) matrix products	**O(log n) multiplies**	O(deg²) GF(2) ops (~10–500 ms)	same	same
Equidistribution	well analysed theory	well analysed theory	none published	1-dim (**/++) / 0-dim (+)	3-dim (**/++) / 2-dim (+)	7-dim (**/++) / 6-dim (+)
Parallelism scale advised	large	very large	mild (2^32 streams)	mild (≈ 2^32 streams)	very large	extreme

- Your application consumes **floats only**: MRG32k3a emits them natively. Its wide integers are assembled from 16-bit chunks and cost four MRG steps.
- You can afford ~3× slower generation than xoshiro.

8.4 When to choose MRG63k3a

Same author, same construction, same stream semantics, sized for 64-bit arithmetic (L'Ecuyer 1999, Table II, fourth entry):

- You want the **MRG stream model but your run consumes integers**. A `UInt64` is two steps instead of four, and the 32-bit chunks it is built from depart from uniformity by 2^-31 where MRG32k3a's 16-bit chunks depart by 2^-16.

Property	Philox4x32-10	Philox4x64-10	Threefry4x32-20	Threefry4x64-20
Block	4 × UInt32	4 × UInt64	4 × UInt32	4 × UInt64
Key (bits) → streams	64 → 2^64	128 → 2^128	128 → 2^128	256 → 2^256
Counter (bits)	128	128	128	128
Substreams per stream	2^64	2^64	2^64	2^64
Words per substream	2^66 × 32 bits	2^66 × 64 bits	2^66 × 32 bits	2^66 × 64 bits
Arithmetic	wide multiply	wide multiply	add–rotate–xor only	add–rotate–xor only
Float64 throughput	93 M/s	210 M/s	84 M/s	147 M/s
Float64 via rand!	157 M/s	359 M/s	100 M/s	207 M/s
Jump to any position	O(1) counter arithmetic	O(1)	O(1)	O(1)
Output uniformity	exact (bijection)	exact	exact	exact
BigCrush	passes (Salmon et al. 2011)	passes	passes	passes

- You want the **longer period** — 2^377 against 2^191 — and the room it buys: 2^127 streams of 2^100 substreams of 2^150 numbers.
- You are **not** bound to the published MRG32k3a stream ids. MRG63k3a's jump distances are this package's own choice; nothing else implements them, so a run cannot be replayed stream-for-stream against another library.
- Prefer MRG32k3a if the run is float-heavy: on Float64 the 63-bit generator is about 17% slower (196 against 236 M/s), because the step costs a 128-bit multiply. It is ahead on every integer type — 93 against 47 M UInt64/s, 199 against 95 M UInt32/s, and 184 against 163 M Float64/s under rand! — and ahead per random *bit* everywhere.

8.5 When to choose a xoshiro/xoroshiro variant

- You want **maximum throughput** or raw 64-bit integers.
- Choose your state size by parallelism needs:
  - Xoroshiro128*: embedded/GPU or few workers only — its 2^64 substreams per 2^32 streams are enough for mild parallelism. Note Xoroshiro128p has a mild Hamming-weight dependency after ~5 TB of output; prefer ss/pp.
  - Xoshiro256*: the sweet spot for general use (Julia itself uses xoshiro256++ as its default RNG). Use + if you generate *only* floating-point numbers from the high bits; otherwise prefer ++/**, whose outputs are fully equidistributed.
  - Xoshiro512*: essentially never needed for period reasons — any period ≥ 2^256 is beyond every imaginable need — but handy when you want many more independent substreams than 2^128 without changing design.

8.6 When to choose PCG64

- You are **porting or checking a NumPy pipeline**. PCG64 is what numpy.random.default_rng() gives you, and the raw 64-bit outputs here match NumPy's exactly for the same state.

- You want the **fastest array fill** in the package: with a single 128-bit word of state, `rand!` reaches 632 M `Float64`/s for `PCG64DXSM`, ahead of every xoshiro. On scalar draws it is slower than xoshiro, at around 500 M/s.
- You need **cheap jumps to arbitrary positions**. The LCG advance is closed-form, so `advance_state!(rng, e, c)` costs `O(log n)` multiplications at any distance, forwards or backwards — against tens to hundreds of milliseconds for the equivalent xoshiro jump.
- Do **not** choose it for large-scale parallelism: with a `2^128` period it offers `2^32` streams, the same order as `Xoroshiro128*`, and there is no equidistribution theory for it. Note also that the stream distances here are odd rather than powers of two — for an LCG that is a correctness requirement, not a style choice; see the Implementation Notes. For many streams, use MRG32k3a, a 256-bit xoshiro, or a counter-based generator.
- Note what this package does *not* do: PCG's own increment-based "streams". See the FAQ.

8.7 When to choose Philox or Threefry

- You need to **address a draw without owning the generator that produced it**: a GPU kernel, a distributed worker, or a re-run that must reproduce replicate `i` of stream `s` without replaying anything. This is the property no recurrence-based generator has.
- You want a **very large number of streams** with no jump computation at all: `next_stream!` on a counter-based generator is an increment.
- You are matching another implementation of the same bijection — Random123, cuRAND, TensorFlow, Intel MKL all ship Philox, and the outputs here agree bit-for-bit with the Salmon et al. test vectors.
- Prefer **Philox4x64-10** as the default counter-based choice: it is the fastest of the four here and the widest-deployed. Prefer **Threefry** when the target hardware has no fast integer multiply, or when you want an add–rotate–xor construction for auditability; the same code reproduces bit-for-bit anywhere.
- Do not choose a counter-based generator for raw single-thread speed: on scalar `Float64` draws they are 3–8× slower than xoshiro. Filling arrays with `rand!` recovers much of that (Philox4x64-10 gains a factor of 1.7) because a whole block lands in the array at once.

8.8 Scrambler choice within a xoshiro family

Scram-bler	Output	Best for
`+`	sum of two state words	float-only workloads using the top bits; lowest 3 bits have low linear complexity
`**`	`rotl(s2·5, 7)·9`	all-purpose; maximal equidistribution (e.g. .NET/Lua default)
`++`	`rotl(s1+s4, k)+s1`	all-purpose; Rust's `SmallRng`, Julia's default

All three share the same transition and therefore the same jump constants: streams/substreams behave identically across scramblers of one family.

8.9 Related work in the Julia ecosystem

Most Julia random-number packages live under the JuliaRandom organisation, and they solve a different problem from this one. They are collections of *generators*; `RandomDataStreams.jl` is a *stream layer* that happens to need generators to apply itself to.

Package	What it provides	Relation to this package
RandomNumbers.jl	PCG, the xorshift family, MT19937 behind `AbstractRNG`	overlapping generators, no stream or substream operations; jump-ahead only for PCG (`advance!`)
Random123.jl	Philox, Threefry, ARS, AESNI	overlapping generators; counter positioning via `set_counter!`, no stream model
StableRNGs.jl	a Lehmer LCG with output stable across Julia versions	complementary; its "stable streams" mean version stability, not a partition of the period
RandomExtensions.jl	distribution objects and `make` specifications for `rand`	orthogonal: it extends what you draw, not where you draw it from
VSL.jl	bindings to Intel MKL's Vector Statistics Library	MKL offers `vslSkipAheadStream` and `vslLeapfrogStream`; the bindings expose `vslNewStream`/`vslNewStreamEx` only
RNGTest.jl	the TestU01 interface	the same C library we validate against, wrapped; we call it directly instead, for the reasons in Validation

The practical consequence: if you need a fast generator, any of these will serve. If you need `n` streams that are provably disjoint, each rewindable to its own checkpoint, with the same four calls whatever the underlying family, that is what this package adds. Should you already depend on one of them, note that the generators here are validated against the same original references — the Salmon et al. test vectors for Philox and Threefry, the `xoshiro.di.unimi.it` C code for the xoshiro families, and NumPy for both PCG variants — so from an identical state they produce identical output. Seeding conventions differ between packages, however, so matching a run means setting the state, not reusing the seed.

8.10 Practical notes

- Seeding a recurrence-based generator: seed states must not be all-zero. For hash-like seeds, initialize with splitmix64 outputs (see Vigna's recommendation). A counter-based generator has no forbidden key in this package — Philox and Threefry are safe for consecutive small keys — but a family whose bijection is weak for structured keys must supply a hashing `stream_key`; see Implementation Notes.
- The package's jumps are **anchored at stream boundaries** (`Cg ← Bg ← Ig`), matching the L'Ecuyer stream model: `next_substream!` always lands at the same place regardless of how far you consumed in the current substream, which makes scenario replay reproducible. Counter-based generators obey the same contract, with the counter's high bits playing the role of `Bg`.
- Every generator here supports `advance_state!(rng, e, c)` with negative distances, so backward jumping is not a differentiator — only its cost is.
- Statistical testing of every generator is described in Validation.

Part IX

FAQ

Chapter 9

FAQ

9.1 Why is an all-zero state forbidden?

For the xoshiro/xoroshiro families the all-zero state maps to itself: the generator would output only zeros forever. The two components of MRG32k3a and MRG63k3a have the same degeneracy (an all-zero component stays zero). Constructors, `srand!`, `set_state!` and `Random.seed!` all reject such a seed with an `ArgumentError`, and `checkseed` (`checkseed63` for MRG63k3a) is the predicate they use, so you can test a seed yourself before handing it over. An integer seed never produces one. PCG and the counter-based generators have no invalid seed at all.

9.2 Are MRG integer outputs uniform over their full width?

No, for neither member of the family, though the two are not equally far from it. MRG32k3a natively produces ~32 bits per draw (`z = p1 - p2` in `[1, m1]`, `m1 = 2^32 - 209`), and wider integers are assembled from 16-bit chunks of that value, whose departure from uniformity is of order 2^-16 per chunk. MRG63k3a produces just under 63 bits (`m1 = 2^63 - 6645`) and assembles 32-bit chunks, at 2^-31 per chunk — the same construction, two orders of magnitude closer to uniform, and half as many steps per word.

Both are fine for indices, shuffling, flags and acceptance tests in simulations, and neither is exactly uniform over 2^64/2^128. Use a xoshiro variant, PCG or a counter-based generator when you need full-width raw words.

9.3 MRG32k3a or MRG63k3a?

Same author, same construction, same stream semantics; the difference is the word size the arithmetic is built for. Take **MRG32k3a** when you need the published, widely implemented parameter set — matching SSJ, RngStreams or another simulation package draw for draw — or when your run consumes floats almost exclusively. Take **MRG63k3a** when the draws are integers (its `UInt64` costs two steps against four, and is far closer to uniform), or when you want the longer period (2^377 against 2^191) and the wider stream spacing that comes with it. Note that the stream and substream distances of MRG63k3a are this package's own choice, since L'Ecuyer published jump matrices for MRG32k3a only.

9.4 Why doesn't `sample(v, k)` work with RandomDataStreams generators?

`sample` comes from StatsBase.jl, not from the Random standard library — even Julia's own RNGs do not provide it without that package. Everything in Random.jl works with RandomDataStreams generators.

9.5 Where did `srand`, `next_stream`, `short_jump`, `long_jump` go?

They mutated their argument despite not ending in `!`; they are deprecated in favour of:

Deprecated	Replacement
`srand(rng/gen, seed)`	`srand!(rng/gen, seed)`
`next_stream(gen)`	`next_stream!(gen)`
`short_jump(rng)`	`short_jump!(rng)`
`long_jump(rng)`	`long_jump!(rng)`

The old names still work but emit a deprecation warning.

9.6 What does an integer seed mean?

The same thing for every generator: it is expanded through splitmix64 and folded into whatever that family accepts as a state or key, so `T(12345)`, `Gen(12345)` and `Random.seed!(T(), 12345)` all agree. A value in the family's own representation — its seed vector, or a `UInt128` for PCG — is taken as the state itself rather than hashed. See Streams & Substreams for the full table.

9.7 Can `rand` return 0 or 1?

No, for every generator and every floating-point type: draws lie in the open interval `(0, 1)`, so `-log(rand(rng))` or a quantile function applied to a draw is always finite. The price is one bit of resolution — 52 bits for a `Float64`, 23 for a `Float32`. In exchange the draws sit on a grid symmetric about 1/2, so `1 - u` is exact and is itself a possible draw, which is what antithetic variates need. See Implementation Notes.

9.8 How do I run truly parallel simulations?

Take one stream per replication first, then parallelise over the replications: `rngs = next_stream!(gen, nrep)`, and replication `r` draws from `rngs[r]`. Streams share no state, so this needs no synchronisation, and since the stream follows the replication rather than the thread, the results do not change with the number of threads.

Never call `next_stream!` on a shared generator object from inside a parallel loop. The generator rewrites the seed of the next stream on every call, so concurrent calls hand out overlapping streams and nothing reports it. For separate processes the seeds have to travel instead; both patterns are in Streams & Substreams.

9.9 Which generator should I pick?

See Generator Comparison. Short answer: `Xoshiro256pp` for general use, `MRG32k3a` for L'Ecuyer-compatible stream semantics (`MRG63k3a` for the same model with 64-bit arithmetic and a longer period), `Philox4x64RNG` when a draw must be addressable by index rather than reached by replaying a state.

9.10 What does "counter-based" buy me?

A counter-based generator computes its output as a keyed bijection of a counter, so the draw at position `i` of substream `j` of stream `s` is a *function* of `(s, j, i)`. Nothing has to be replayed to reach it. That makes three things easy that are awkward otherwise: handing a stream identifier to a worker or a GPU kernel that has no generator object, jumping to an arbitrary position in constant time, and re-running one replicate of one stream without re-running anything else. The cost is speed: on scalar draws Philox and Threefry are several times slower than xoshiro, though filling arrays with `rand!` recovers much of the gap.

9.11 Philox or Threefry?

`Philox4x64RNG` is the faster of the two here and the one other libraries ship (Random123, cuRAND, TensorFlow, Intel MKL), so prefer it unless you have a reason not to. `Threefry4x64RNG` uses only additions, rotations and xors — no multiplication — which makes it a better fit for hardware without a fast wide multiply, and easier to audit.

9.12 Is it safe to use 1, 2, 3, ... as counter-based stream keys?

For Philox and Threefry, yes: ten (resp. twenty) rounds of encoding absorb the structure of consecutive small keys, as reported by Salmon et al. (2011) and discussed by L'Ecuyer et al. (2021, Sec. 3). This is *not* a property of counter-based generators in general — for the Squares generator, the key 0 gives an identically zero output — so a new family whose bijection is weak for structured keys must override `stream_key` with a schedule that hashes the seed. See Implementation Notes.

9.13 Can I reproduce cuRAND's Philox and MRG32k3a streams?

Yes, word for word. cuRAND's `Philox4_32_10` is this package's `PhiloxRNG`: the seed is the key, its low 32 bits first, and cuRAND's subsequence is the package's substream — the high half of the counter — with the offset counting words into it:

```
rng = PhiloxRNG((s % UInt32, (s >> 32) % UInt32))   # cuRAND's seed s
for _ in 1:k
    next_substream!(rng)                             # curand_init(s, k, 0)
end
rand(rng, UInt32)                                    # the first curand(&state)
```

The same holds for the host API, whose default ordering for Philox takes four words from each of 65,536 subsequences in turn. The test suite checks words that cuRAND generated itself, through both APIs, at subsequences up to `2^64 − 1`.

What carries over is the 32-bit words, not what is built on them. cuRAND's `curand_uniform` returns `x·2^-32 + 2^-33` in single precision, which is `1.0f0` for the largest word: its floats lie in `(0, 1]`, this package's in `(0, 1)`. Its normals are Box-Muller, this package's inversion. A pipeline that uses cuRAND's distributions therefore does not reproduce; one built on its words does, and a kernel of your own reaches any of them through `philox4x32_10` and `philox4x32_counter`.

cuRAND's `MRG32k3a` is this package's `MRG32k3a` too. Seed 0 is L'Ecuyer's published state, the one `MRG32k3a()` starts from; another seed sets each component to `12345·x mod m`, x being the seed's low half XOR `0x55555555` or its high half XOR `0xAAAAAAAA`, as `curand_init` does. Its subsequence k is the package's substream k, `k·2^76` steps in — `next_substream!` from `MRG32k3a()` for seed 0 — and the host API's default ordering takes output n from step `n ÷ 81920` of subsequence `n mod 81920`. Here what carries over is the combination z in `[1, m1]`: cuRAND's doubles are `z/m1`, which is `1.0` for `z = m1`, the package's `z/(m1 + 1)`, and its 32-bit integers are `z` scaled to `2^32 − 1`, not the package's.

9.14 Can I reproduce a NumPy pipeline?

Partly. `PCG64` here produces exactly the same raw 64-bit outputs as `numpy.random.default_rng()`'s bit generator for the same 128-bit state, and `PCG64DXSM` matches NumPy's `PCG64DXSM`. What does *not* transfer is how NumPy gets to that state: `SeedSequence` turns a user seed into the state through its own hashing, and that specification is not implemented here. So you can reproduce a NumPy stream if you carry the state across, not if you carry only the seed.

The distributions differ too. `rand(rng)` and `numpy`'s `random()` consume the same bits but not necessarily in the same way, and `randn` draws by inversion where NumPy uses a Ziggurat. Only the raw generator output is guaranteed to agree.

9.15 Why can't I choose PCG's increment to get more streams?

Because it does not give independent streams. Every odd increment gives a full-period orbit, but for half of all pairs of odd increments the two orbits are the same state sequence shifted by a constant — an adversarial pair can be built whose outputs differ by an average of 2 bits out of 64. Most pairs are harmless, but no published criterion separates them, so the package fixes the increment and derives streams from jumps instead. The algebra is in the Implementation Notes. NumPy takes the same practical position: its recommended way to parallelise is `SeedSequence.spawn()`, not the increment.

9.16 Does this package run on the GPU?

For the counter-based generators, yes. With CUDA.jl loaded, `rand!` fills a `CuArray{Float64}`, `CuArray{Float32}` or `CuArray{Float16}` on the device, bit for bit what the same call gives on the CPU from the same position, and leaves the generator object — which stays on the host — where the CPU fill would. `randn!` and `randexp!` do the same for normal and exponential variates, which every generator draws by inversion: the GPU's values are the CPU's. The recurrence-based generators assign and navigate streams on the host only. See Streams & Substreams.

9.17 Why does `randn` use inversion rather than the Ziggurat?

Because the Ziggurat breaks what this package's streams are for. It takes a variable number of uniforms per normal, and builds a normal from different bits of one word, so reflecting a uniform does not negate a normal, two systems that share uniforms need not share normals, and a quasi-Monte Carlo point does not map to one normal per coordinate. Inversion, one uniform per normal through Φ^{-1}, has none of these problems, at a cost in speed. The comparison is tabulated in Normal variates and variance reduction.